

RTL Design Sherpa

amber — Blocking MESI Snoopy L1 Cache Hardware Architecture Specification 0.5

October 6, 2026

Table of Contents

1 Amber Has Index....13

2 amber MAS Index....14

2.1 Document Organization....14

2.1.1 Front Matter....14

2.1.2 Chapter 1: Overview....14

2.1.3 Chapter 2: Functional Blocks....14

2.1.4 Chapter 3: Interfaces....14

2.1.5 Chapter 4: MonBus Observation Map....15

2.1.6 Chapter 5: Verification Bring-Up....15

2.1.7 Chapter 6: Configuration....15

2.2 Quick Reference....15

2.2.1 Core Blocks....15

2.2.2 Top Modules....16

2.3 Related Documentation....16

3 amber — Product Requirements (DRAFT)....18

3.1 1. Purpose....18

3.2 2. Background....18

3.3 3. Decisions that pick the design....19

3.4 4. Success criteria....28

3.5 5. Non-goals....29

3.6 6. Where the block sits....29

4 Global Hard Requirements for RTL Design Sherpa....30

4.1 Overview....30
4.2 Category 1: RTL Standards....30
4.2.1 1.1 Reset Handling (MANDATORY - NO EXCEPTIONS)....30
4.2.2 1.2 FPGA Synthesis Attributes (MANDATORY)....31
4.2.3 1.3 Array Syntax Standard (MANDATORY)....32
4.2.4 1.4 SRAM Module Standards (MANDATORY)....32
4.2.5 1.5 Reset Convention (STANDARD)....33
4.3 Category 2: Testbench Architecture....33
4.3.1 2.1 TB Location Requirements (MANDATORY)....33
4.3.2 2.2 Three Mandatory TB Methods (MANDATORY)....34
4.3.3 2.3 TBBase Inheritance (MANDATORY)....34
4.3.4 2.4 Three-Layer Architecture (MANDATORY)....35
4.3.5 2.5 Queue-Based Verification (PREFERRED)....36
4.4 Category 3: Test Standards....36
4.4.1 3.1 Test Naming Convention (MANDATORY)....36
4.4.2 3.2 Test Pattern - CocoTB + Pytest (MANDATORY for projects/components/)....37
4.4.3 3.3 100% Success Requirement (MANDATORY)....37
4.4.4 3.4 Simulation Timestamps in Logs (MANDATORY)....38
4.5 Category 4: Framework Usage....39
4.5.1 4.1 Search Before Creating (MANDATORY)....39
4.5.2 4.2 BFM Usage for Protocols (MANDATORY for RAPIDS)....39
4.6 Category 5: Documentation....40
4.6.1 5.1 No Emojis in Technical Docs (MANDATORY)....40

4.7 Category 6: Project-Specific Requirements....40
4.7.1 6.1 Commit Messages Carry No Attribution Trailer....40
4.7.2 6.2 STREAM Tutorial Focus....40
4.8 Summary by Priority....41
4.8.1 P0 - Critical (Enforcement Required)....41
4.8.2 P1 - High Priority (Strong Recommendation)....41
4.8.3 P2 - Standard Practice (Best Practices)....41
4.8.4 P3 - Project-Specific....41
4.9 Compliance Checklist....42
4.10 Enforcement....42
4.11 References....43
5 Document Information....44
5.1 Related documents....44
5.2 Terminology....45
5.3 Revision history....46
5.4 How to read this book....46
6 Purpose and Scope....48
6.1 Purpose....48
6.2 Scope....48
6.3 Why a blocking cache matters....48
7 Document Conventions....50
7.1 Signal naming....50
7.2 Numbering and bases....50

7.3 Citation discipline....50
7.4 Diagram conventions....50
7.5 Voice....51
8 Definitions and Acronyms....52
9 Use Cases....54
9.1 Primary use case: a correct, observable, provable L1 cache baseline....54
9.2 Secondary use cases....54
9.3 What amber is not designed for....54
10 Key Features....55
11 System Context....57
11.1 Where amber sits....57
11.2 Pair rig (amber top)....57
11.3 Onyx rig (amber_ace top)....57
11.4 Shared core, different tops....57
12 Block Diagram....58
12.1 Module roles....58
13 Top Structure and the Two Rigs....60
13.1 No rig parameter....60
13.2 amber — pair-rig top....60
13.3 amber_ace — onyx-rig top....60
13.4 What is inside amber_core....60
14 Data Flow: Hits, Misses, Fills, and Drains....62
14.1 Hit path....62

14.2 Miss path....62
14.3 Snoop path....62
14.4 Open shape: memory-side masters (D3)....63
14.5 Open shape: array construction (D11)....63
15 Control FSM: Blocking Pipeline and Pending-Fill Bypass....64
15.1 The blocking pipeline....64
15.2 Pending-fill bypass register....64
15.3 FSM oracle derivation....65
16 Coherence FSM and the Snoop Responder....66
16.1 MESI state handling....66
16.2 Snoop flow....68
16.3 Ordering convention....69
16.4 Book-informed snoop behavior....69
17 Arbitration and the Victim Bypass....71
17.1 Dual-port array arbitration....71
17.2 Victim bypass....71
17.3 Liveness target....71
18 CPU-Side GAXI Slave....72
18.1 Interface decision....72
18.2 Channel structure....72
18.3 No register block....73
19 Fabric-Side AXI4 Masters....74
19.1 Where they live....74

19.2 Burst behavior....74
19.3 Port summary....74
19.4 D3: decided....74
20 Fabric-Side ACE Masters and Coherent Issue....76
20.1 Onyx-rig fabric....76
20.2 amber_ace_issue....76
20.3 House-module pin contracts — not restated here....76
20.4 Outside the onyx D2 subset....77
21 Snoop Port: AC/CD/CR Adapter....78
21.1 Adapter purpose....78
21.2 House-module pin contract — not restated here....78
21.3 Ordering and the family convention....78
21.4 RACK/WACK question (open interface issue)....78
21.5 Bus-agnostic core boundary....79
22 Observation: MonBus through amber_monlite....80
22.1 Observation decision....80
22.2 MonBus packet content....80
22.3 Observer cost....80
23 amber_pkg and Build-Time Geometry....82
23.1 Parameter philosophy....82
23.2 Core parameters....82
23.3 Derived quantities....82
23.4 Package contents....83

23.5 Rig is not a parameter....83
24 Clocking and Reset....84
24.1 Clock domain....84
24.2 Reset....84
24.3 Reset sequence....84
24.4 Clock gating....84
25 Verification Strategy....85
25.1 Pattern-B cocotb grids....85
25.2 Golden-model parity....85
25.3 Formal targets....85
25.4 Python reference model....85
25.5 MonBus coverage....85
26 Characterization....87
26.1 Board targets....87
26.2 Sweep dimensions....87
26.3 Reported metrics....87
26.4 Critical-path expectation....87
27 Future Hooks....89
27.1 Recorded future work, not blockers....89
27.2 Closing D3 and D11....91
28 amber — Pre-Hardware Architecture Specification (Pre-HAS)....92
28.1 1. Intent of this document....92
28.2 2. Feature sketches....92

28.2.1 2.1 Core datapath — the blocking bet....92
28.2.2 2.2 Coherence....93
28.2.3 2.3 Fabric side — where the two deployment rigs meet....94
28.2.4 2.4 Observation (D8 DECIDED — sketched, not re-decided)....95
28.2.5 2.5 Verification & formal surface (D9 OPEN — direction proposed)….95
28.2.6 2.6 Cost & characterisation (PRD criterion 4)....95
28.3 3. Proposed block architecture....96
28.4 4. Interface sketches (shapes, not pinouts)....97
28.5 5. Parameter sketch (illustrative — feeds the DV matrix)....98
28.6 6. Open questions this sketch surfaces....99
28.7 7. What the full HAS will add....100
28.8 8. Cross-references....100
29 jet — Product Requirements (DRAFT)....102
29.1 1. Purpose....102
29.2 2. Decisions jet adds to amber’s....102
29.3 3. Success criteria (additions to amber’s)....104
29.4 4. Non-goals (for jet, beyond amber’s)....104
30 onyx — Product Requirements (DRAFT)....105
30.1 1. Purpose....105
30.2 2. Background....105
30.3 3. Decisions that pick the design....106
30.4 4. Design invariants (proposed)....125
30.5 5. Success criteria....126

30.6 6. Non-goals....126
30.7 7. Where the block sits....126
30.8 8. References....127
31 AMBA ACE — Working Interface Definition for cache-ip....128
31.1 1. What ACE is....128
31.2 2. Channel inventory....128
31.2.1 2.1 Coherent master → manager (front side, per master)....128
31.2.2 2.2 Manager → cache, cache → manager (snoop side, per cache)....129
31.3 3. Transaction types (the onyx D2 research subset in bold)....130
31.4 4. The CRRESP decision bits....131
31.5 5. ACE-Lite....132
31.6 6. What this family implements, and what it deliberately does not....132
31.7 7. Sources....132
32 gem5 Ruby protocol specs (reference extract)....134
33 Cache Associativity Simulator....135
33.1 Files....135
33.2 Model API....135
33.3 Running tests....136
33.4 UI features....136

List of Figures

No figures in this document.

List of Tables

Table 1: Table 0.1: Document information....44
Table 2: Table 0.2: Related documents....44
Table 3: Table 0.3: Revision history....46
Table 4: Table 1.0: Definitions and acronyms....52
Table 5: Table 2.0: Key features and their decisions....55
Table 6: Table 2.1: Module roles....58
Table 7: Table 3.0: MESI snoop CRRESP matrix — DT=DataTransfer, PD=PassDirty, IS=IsShared, WU=WasUnique. Snoop encodings are the six IHI0022 AC types; CleanUnique and MakeUnique are transactions a master issues (the onyx D2 subset), not snoop encodings, so they do not appear here....66
Table 8: Table 4.0: CPU-side GAXI slave channels....72
Table 9: Table 4.1: Plain AXI4 channels in the pair rig....74
Table 10: Table 4.2: Cache events to ACE transactions....76
Table 11: Table 4.6: MonBus event classes....80
Table 12: Table 5.0: Core elaboration parameters....82
Table 13: Table 6.0: Characterization sweep dimensions....87
Table 14: Table 6.1: Future hooks....88

1 Amber Has Index

Generated: 2026-10-11

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 amber MAS Index

Version: 0.5 **Date:** 2026-10-06 **Purpose:** Micro-architecture specification for the amber blocking MESI snoopy L1 cache

Scope: Cycle-level behavior, per-module micro-architecture, and the pre-RTL signal-contract / kmap workbook for `amber` (pair-rig top) and `amber_ace` (onyx-rig top). The binding architecture lives in the amber HAS; this book closes the implementation decisions the HAS deferred.

2.1 Document Organization

Note: All chapters linked below for automated document generation.

2.1.1 Front Matter

- Document Information

2.1.2 Chapter 1: Overview

- Architecture and What the MAS Covers
- Top-Level Port List
- Clocks and Reset

2.1.3 Chapter 2: Functional Blocks

- amber_control: Blocking Pipeline FSM
- Pending-Fill Bypass Register
- amber_tag_array and amber_data_array
- amber_repl: Replacement Policy Engine
- amber_victim: Depth-1 Victim Buffer
- amber_fill and amber_drain
- amber_snoop_resp: Snoop Responder Adapter
- amber_ace_issue: Coherent Transaction Mapping
- amber_cpu_frontend and amber_monlite

2.1.4 Chapter 3: Interfaces

- CPU GAXI Request/Response Timing
- Fabric AXI4 Master Timing
- Fabric ACE Master Timing

- Snoop AC/CD/CR Timing and Ordering
- MonBus Event Emission Points

2.1.5 Chapter 4: MonBus Observation Map

- Event Encodings and Emission Points
- Drop-and-Count Behavior and monbus_tally_axil

2.1.6 Chapter 5: Verification Bring-Up

- What the K-maps Prove
- RTL Landing Diff Procedure
- gem5 FSM Cross-Check

2.1.7 Chapter 6: Configuration

- Elaboration Configurations
- DV Matrix Dimensions

2.2 Quick Reference

2.2.1 Core Blocks

Module	File	Purpose	Status
`amber_control`	`rtl/fub/amber_control.sv`	Blocking pipeline FSM, state encoding, miss/replay sequencing	Pre-RTL contract
`amber_tag_array`	`rtl/fub/amber_tag_array.sv`	Dual-port `sdpram_core` tag + MESI state store	Pre-RTL contract
`amber_data_array`	`rtl/fub/amber_data_array.sv`	Dual-port `sdpram_core` data store	Pre-RTL contract
`amber_repl`	`rtl/fub/amber_repl.sv`	LRU / tree-PLRU / FIFO / RANDOM replacement	Pre-RTL contract
`amber_victim`	`rtl/fub/amber_victim`	Depth-1 dirty victim buffer	Pre-RTL contract

Module	File	Purpose	Status
	.sv		
amber_fill	rtl/fub/amber_fill.sv	Line-fill read master interface	Pre-RTL contract
amber_drain	rtl/fub/amber_drain.sv	Dirty victim write-back interface	Pre-RTL contract
amber_snoop_resp	rtl/fub/amber_snoop_resp.sv	ACE snoop adapter, CR/CD ordering	Pre-RTL contract
amber_ace_issue	rtl/fub/amber_ace_issue.sv	Cache events → ACE transactions	Pre-RTL contract
amber_cpu_frontend	rtl/fub/amber_cpu_frontend.sv	GAXI slave request/response latch	Pre-RTL contract
amber_monlite	rtl/fub/amber_monlite.sv	Drop-and-count MonBus observer	Pre-RTL contract

2.2.2 Top Modules

Module	File	Purpose	Status
amber	rtl/top/amber.sv	Pair-rig top, AXI4 memory + ACE snoop	Pre-RTL contract
amber_ace	rtl/top/amber_ace.sv	Onyx-rig top, full ACE coherent issue	Pre-RTL contract
amber_core	rtl/macro/amber_core.sv	Shared core (both rigs)	Pre-RTL contract

2.3 Related Documentation

Document	Where	Relationship
amber PRD	../../PRD.md	**binding requirements** — all D-rows and success criteria originate here
amber HAS	../amber_has/	architecture

Document	Where	Relationship
	amber_has_index.md	specification this MAS implements
amber Pre-HAS	../amber_has/amber_prehas.md	original feature sketches
K-map / signal-contract workbook	../gen_amber_contracts_kmaps.py	pre-RTL combinational-contract sheets and kmaps
jet PRD	../../../jet-mesi-l1/PRD.md	lockup-free follow-on; amber must not pre-decide jet's J-rows
onyx PRD	../../../onyx-ace-ccu/PRD.md	onyx D7 defines the ACE-shaped snoop port; onyx D2 defines the coherent-transaction subset
AMBA ACE Interface Definition	../../../References/AMBA_ACE_Interface_Definition.md	in-repo ACE contract, verified against IHI 0022H
gem5 Ruby protocols	../../../References/gem5-ruby-protocols/README.md	golden-protocol source for the Python reference model and FSM oracles
cache simulator	../../../../../../bin/apps/cache_sim/README.md	golden parity model for LRU/FIFO/RANDOM
Global Requirements	../../../../../../GLOBAL_REQUIREMENTS.md	reset macros, array syntax, no bespoke SRAM, filelist registry

Last Updated: 2026-10-06 **Maintained By:** amber architecture

3 amber — Product Requirements (DRAFT)

Version: 0.5 (draft, 2026-10-06) **Status:** nine of the eleven decision rows record a name and a date; D3 (memory-side, now shaped by the two-top structure) and D11 (arrays, house primitives by rule) remain OPEN. A decision becomes DECIDED when it records a name and a date, the way the reed-solomon PRD does it. v0.2: D8 decided — `*_monlite` observation wrappers (never `_mon` on the measured paths). v0.3: D4 decided — snoop transport is ACE-shaped (AC/CD/CR) per the onyx D7 port contract, family decision 2026-10-05. The snoop-filter (directory-lite) half of D4 stays deferred and is reopened as onyx D3. v0.4: D2 decided (Sean, 2026-10-06) — CPU-side interface is a **GAXI slave**. House BFM/monitor coverage and skid/FIFO plumbing already exist, and STREAM can attach as first real consumer without an adapter. v0.5 (Sean, 2026-10-06): D1, D5, D6, D7, D9, D10 decided in one working session — geometry 32 KiB/64 B/4-way center + a tiny 16-set/2-way formal config; write-back + write-allocate; plain MESI on a 3-bit state field; replacement {LRU, tree-PLRU, FIFO, RANDOM} with LRU default (tree-PLRU the timing fallback, no sim golden model — future sim extension recorded); verification = Pattern-B grids + control-layer SymbiYosys at tiny geometry + cache_sim parity on LRU/FIFO/RANDOM + FSM oracles derived from the gem5 Ruby MESI_Two_Level SLICC tables; first consumer = TB masters bring-up, pair rig the gated deliverable, STREAM attach the documented future hook. v0.6 (Sean, 2026-10-07): D3 and D11 decided, unblocking the cache-core RTL. D3 — memory-side is **AXI4 read/write masters on the house `axi4_master_rd`/`axi4_master_wr` wrappers (the stream/rapids pattern)**, observed through `*_monlite` per D8: the monlite decision needs monitored memory-side channels, the masters match the house BFM/monitor coverage that D2 chose GAXI for on the CPU side, and the pair rig's shared memory hangs off them through house fabric. The simple-SRAM-port bring-up shortcut was set aside as a rewrite-in-waiting. D11 — **DECIDED in the documented shared-primitives direction: `sdpram_core`-based tag/data stores (house no-bespoke-SRAM rule)** + **`gaxi_fifo_sync` house FIFOs for the pending/fill queues**; FPGA attributes per GLOBAL_REQUIREMENTS.md.

3.1 1. Purpose

A parameterised, blocking, snoopy MESI L1 data cache for the Sherpa research/teaching flow: correct against a software golden model, provable with SymbiYosys, observable through the MonBus monitor fabric, and characterised for FPGA cost like every other component in this repo. It is the first cache-ip IP and the deliberate baseline for jet, the lockup-free follow-on.

3.2 2. Background

A blocking cache freezes on a miss: the pending request (and everything behind it) waits for the fill to complete. That is poor for performance and excellent for correctness work — one outstanding transaction means no MSHR bookkeeping, no hit-under-miss hazards, no merge logic. MESI (Modified / Exclusive / Shared / Invalid) is the coherence layer on top: each line carries a state that says who may write it and who must be told when they hold a stale copy. Snoopy coherence broadcasts probes on a shared bus; every cache watches and answers.

Blocking and MESI are orthogonal, which is the design bet: amber nails the coherence protocol with trivial miss handling; jet keeps the protocol and upgrades the miss handling. The pair then answers the classic question with measured, on-board data: what does lockup-freedom actually buy?

Repo anchors the design must respect: resets on the `ALWAYS_FF_RST` macros, storage on the shared `sdpram_core` / gaxi FIFO primitives (no bespoke SRAM, no reset on memories), `[DEPTH]` array syntax, valid/ready streaming contracts, and the monitor/MonBus observation fabric from `rtl/amba/`.

3.3 3. Decisions that pick the design

#	Decision	Candidates	What it drives
D1	Cache geometry	**DECIDED 2026-10-06 (Sean): 32 KiB / 64 B lines / 4 ways (128 sets) as the default center, fully elaboration-parameterised (4–32 KiB, 32–64 B, 2–8 ways), plus a reserved tiny config — 16 sets / 2 ways / 64 B — for SymbiYosys tractability.** Board BRAM does not bind (~9 of 135 36Kb RAMs on Nexys A7 at the default), so geometry serves the DV matrix and the proofs, not the board	tag/data array dimensions, the whole DV matrix
D2	CPU-side interface	**DECIDED 2026-10-06 (Sean):**	front-end FSM, adapter surface,

#	Decision	Candidates	What it drives
		GAXI slave. Chosen over the plain valid/ready native port and the AXI4 slave: house BFM/monitor coverage and skid/FIFO plumbing already exist, and STREAM can attach as first real consumer without an adapter. Must express single-beat reads, write-allocate fills, and (if write-through is chosen) store-without-allocate	first consumer
D3	Memory-side interface	**DECIDED 2026-10-07 (Sean): AXI4 read/write masters on the house `axi4_master_rd`/`axi4_master_wr` wrappers (stream/rapids pattern), `*_monlite` observation per D8.** Chosen over GAXI (D2 already spent that on the	fill/drain engines, burst behavior on misses

#	Decision	Candidates	What it drives
		CPU side) and over the simple SRAM bring-up port (a rewrite-in-waiting); matches D8's monitored-paths requirement and the pair-rig shared-memory hook	
D4	Snoop transport	**DECIDED 2026-10-05 (Sean): ACE-shaped transport — amber implements the AC/CD/CR snoop channels exactly as onyx D7 defines them (family decision: cache-ip is ACE-shaped), staying bus-agnostic internally behind a thin adapter.** Chosen over a custom dedicated snoop bus and over snooping on the AXI4 fabric directly; the ACE-lite-shaped candidate was set	the coherence bus, formal surface, gate count

#	Decision	Candidates	What it drives
		aside because a snoop responder must drive CR responses and CD data — ACE-Lite carries no snoop channels at all. The snoop-filter (directory-lite) sub-question stays deferred, reopened as onyx D3.	
D5	Write policy	**DECIDED 2026-10-06 (Sean): write-back + write-allocate.** Write-through/no-allocate deletes the coherence traffic the research exists to measure — dirty snoop transfers (PassDirty), WriteBack drains, onyx's memory-read elision — and leaves MESI's M state with no job; it stays available as a bring-up mode only	M-state logic, memory traffic, formal targets
D6	MESI variant	**DECIDED 2026-10-06 (Sean):**	state machine count, snoop

#	Decision	Candidates	What it drives
		plain MESI on a 3-bit state field. MOESI's O-state is a later elaboration upgrade (encoding headroom reserved now, ~512 extra tag bits); its win — dirty→shared without a memory read — is a measurement for the pair rig, not a v1 requirement	response matrix
D7	Replacement policy	**DECIDED 2026-10-06 (Sean): policy set {LRU, tree-PLRU, FIFO, RANDOM}, default LRU.** LRU/FIFO/RANDOM match the cache simulator exactly (`POLICIES = ['LRU','FIFO','RANDOM']`), so golden-model parity covers every policy both sides implement; **tree-PLRU is carried as the**	policy engine, cross-check fidelity

#	Decision	Candidates	What it drives
		timing-tight fallback — the one policy with no sim golden model, cross-checked by RTL self-checks today and a future cache_sim extension (recorded, not silently assumed). Policy stays an elaboration parameter so the sim↔RTL cross-check holds per policy	
D8	Observation	**DECIDED 2026-10-04 (Sean): `*_monlite` wrappers, not the heavyweight `_mon`.** MonBus taps for hits, misses (class split if the model supplies it), snoops, evictions, and state transitions, observed through the `axi4_monlite` / `axil4_monlite` family — same	perf counters, board capture, DV scoreboards

#	Decision	Candidates	What it drives
		128-bit MonBus packets, UNIT/AGENT ids, and host tooling as `_mon`, ~1/5 the monitor gates, and a drop-and-count policy that never stalls the port. Rationale for a research cache: the observer must not perturb what it measures — a `_mon` wrapper that gates the port when its tracking tables fill would corrupt the very miss-latency numbers amber and jet exist to produce. The events D8 lists are emitted as MonBus packets and tallied by the standard agents (`monbus_tally_axil`); heavyweight `_mon` stays available as a DV cross-check, never on the measured paths.	
D9	Verification	**DECIDED 2026-**	DV structure,

#	Decision	Candidates	What it drives
	strategy	**10-06 (Sean): Pattern-B cocotb GATE/FUNC/FULL grids over the geometry × policy × rig matrix (ACE BFMs: cocotb-framework 1.2.0); SymbiYosys scoped to the control layer — `amber_control`, `amber_snoop_resp`, the pending-fill bypass, the victim-buffer handoff — proven at the tiny 16-set/2-way geometry; cache_sim trace-replay parity across LRU/FIFO/RANDOM. The Python reference model and the FSM oracles derive from the gem5 Ruby `MESI_Two_Level` SLICC tables in `../References/gem5-ruby-protocols/` —**	formal areas, CI time

#	Decision	Candidates	What it drives
		`L1cache.sm`'s enumerated transient states are the derivation source for the `amber_pkg` state encoding, and every stable and transient transition is cross-checked against an executable protocol spec, not prose	
D10	First consumer	**DECIDED 2026-10-06 (Sean): standalone TB masters for bring-up; the gated deliverable is the pair rig — two ambers + shared memory, the coherence research rig.** STREAM attach is documented as the future integration hook (D2's GAXI choice is what makes it cheap later); the onyx rig is sequenced per	integration scope, what "done" means

#	Decision	Candidates	What it drives
		onyx D10 — onyx waits behind the pair rig	
D11	Data/tag arrays	**DECIDED 2026-10-07 (Sean), in the documented shared-primitives direction: `sdpram_core`-based tag/data stores** + **`gaxi_fifo_sync` house FIFOs for the pending/fill queues**; FPGA attributes per `../../../../GLOBAL_REQUIREMENTS.md`. House no-bespoke-SRAM rule applies	area/timing headroom, the no-bespoke-SRAM rule

3.4 4. Success criteria

1. **Golden-model parity.** Given the same trace + policy + geometry, amber's hit/miss/miss-class counts match cache_sim exactly across a regression trace suite.
2. **Proofs.** SymbiYosys: no stale data is ever served after an external write; the coherence protocol cannot deadlock under fair arbitration.
3. **Observation.** Every event class above visible on MonBus in simulation, and captured on at least one FPGA board.
4. **Cost.** Synthesis/implementation numbers (LUT/FF/RAM, fmax) on Nexys A7 and/or Genesys 2, recorded in `docs/` the way pumice characterised.

3.5 5. Non-goals

Non-blocking / lockup-free miss handling (jet), directory-based coherence, L2/L3 or multi-level inclusion policies, ECC on cache arrays, virtual memory and TLB interaction, multi-port CPU issue. Each is a future IP or a future decision, not an amber requirement.

3.6 6. Where the block sits

Research cache on the AMBA fabric: CPU-side slave (D2), memory-side AXI4 masters (D3), snoop port toward onyx / peer caches (D4 — ACE-shaped AC/CD/CR, decided 2026-10-05), MonBus observation (D8). Standalone-testable with TB masters before any consumer exists (D10).

4 Global Hard Requirements for RTL Design Sherpa

Version: 1.0 **Created:** 2025-10-31 **Purpose:** Consolidated mandatory requirements extracted from all CLAUDE.md files

---

4.1 Overview

This document consolidates all **MANDATORY** requirements found across repository CLAUDE.md files. These are **HARD REQUIREMENTS** that must be followed - not suggestions or guidelines.

Sources Analyzed: - Root `/CLAUDE.md` - `projects/components/CLAUDE.md` - `projects/components/dma-ip/rapids/CLAUDE.md` (cited below as `rapids/CLAUDE.md`) - `projects/components/dma-ip/stream/CLAUDE.md` (cited below as `stream/CLAUDE.md`) - `rtl/amba/CLAUDE.md` (cited below as `amba/CLAUDE.md`) - `rtl/common/CLAUDE.md` - `CocoTBFramework/CLAUDE.md` - the DV framework guide, now maintained in the separate RTLDesignSherpa-DV repo (the framework is editable-installed into the venv)

4.2 Category 1: RTL Standards

4.2.1 1.1 Reset Handling (MANDATORY - NO EXCEPTIONS)

Requirement: ALL RTL in projects/components/ MUST use standardized reset macros

Pattern:

```
`include "reset_defs.svh"

`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
    end else begin
        r_state <= w_next_state;
    end
)
```

NOT Allowed:

```
// WRONG - Manual reset handling
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_state <= IDLE;
    end else begin
        r_state <= w_next_state;
```

```
    end
end
```

Applies to: ALL synthesisable RTL – `rtl/` and `projects/` alike. **Source:** `projects/components/CLAUDE.md` Rule #0, widened 2026-09-06 (Sean: "All flops should use the global macro").

Reset is asynchronous on assertion. Always. `ALWAYS_FF_RST` used to be conditional on `USE_ASYNC_RESET` and defaulted to SYNCHRONOUS, so `make lint` (which set the define) and simulation/synthesis (which did not) disagreed about what the design was. The define is now a no-op; passing it is harmless and changes nothing. Deassertion still has to be synchronised externally.

The one exception, and it is structural: `rtl/cdc/reset_sync.sv`. It is active-HIGH, which the macro cannot express without `RESET_ACTIVE_HIGH` set globally, so it keeps a raw `always_ff`. Documented at the flop itself. If you add another, document it there too and say WHY – "it has always been like that" is not a reason.

"Macro-free fork" is not a reason. Two trees claimed it and neither survived contact with evidence. `timing_characterization/rtl/asic_only/` was a 37-file duplicate of `rtl/` differing ONLY in reset spelling; synthesised through Yosys + slang against ASAP7 it produced a byte-identical mapped netlist (396 cells, 97 flops, every cell type matching), and the flow reading it still passed `-DUSE_ASYNC_RESET`, a define meaningful only to the macro tree it was cloned from. `formal/cdc/cdc_handshake/cdc_handshake_formal.sv` claimed Yosys could not take the macro, while 92 other .sby units stage `reset_defs.svh` and read it with `-Iincludes`; its own .sby had simply never staged it, and it now proves PASS with the macro. The macros exist to serve ASIC and FPGA both – that is the whole point of them – so a second copy of a module to spell reset differently is duplication, not portability. Deleted 2026-09-07.

Status: `rtl/` converted 2026-09-06 (45 blocks / 20 files; the claim that it was "already compliant" was wrong). `projects/components/utility-ip/converters/` done as projects/components/utility-ip/converters BUG-007 (was CONV-009). Hand-written RTL still outstanding elsewhere under `projects/`, plus ~928 blocks in GENERATED files, which must be fixed at their generators and never in the emitted `.sv`.

4.2.2 1.2 FPGA Synthesis Attributes (MANDATORY)

Requirement: ALL memory arrays MUST have FPGA synthesis hints

Pattern:

```
`ifdef XILINX
    (* ram_style = "auto" *)
`elsif SYNTH_PRAGMA
    /* synthesis ramstyle = "AUTO" */
`endif
logic [DATA_WIDTH-1:0] mem [DEPTH];
```

Attributes: - Xilinx: `ram_style = "auto" | "block" | "distributed" | "ultra"` - Quartus: `ramstyle = "AUTO" | "M20K" | "MLAB" | "logic"`

Applies to: All memory array declarations **Source:** `projects/components/CLAUDE.md` Rule #1

4.2.3 1.3 Array Syntax Standard (MANDATORY)

Requirement: Use `[DEPTH]` NOT `[0:DEPTH-1]`

Pattern:

```
// CORRECT
logic [DATA_WIDTH-1:0] mem [DEPTH];

// WRONG
logic [DATA_WIDTH-1:0] mem [0:DEPTH-1];
```

Reason: Iverilog compatibility, cleaner syntax **Applies to:** All unpacked array declarations **Source:** `projects/components/CLAUDE.md` Rule #2

4.2.4 1.4 SRAM Module Standards (MANDATORY)

Requirement: SRAM modules DO NOT have reset ports

Pattern:

```
module simple_sram #(
    parameter int DATA_WIDTH = 32,
    parameter int DEPTH = 1024
) (
    input  logic clk,
    // NO rst_n port!
    // ... other ports
);
```

Reason: - Real SRAM chips don't have reset pins - FPGA block RAM doesn't reset contents - Initialize via writes during system init

Applies to: All SRAM/memory modules **Source:** `projects/components/CLAUDE.md` Rule #3

4.2.5 1.5 Reset Convention (STANDARD)

Requirement: ALWAYS use active-low asynchronous reset

Signal Names: - `rst_n` - Active-low reset (projects/components/) - `aresetn` - Active-low asynchronous reset (AMBA standard) - `i_rst_n` - Active-low reset (rtl/common/)

NOT Allowed: - `rst` (positive reset) - `reset` (ambiguous polarity)

Applies to: All RTL modules **Source:** Root `/CLAUDE.md`, `rtl/common/CLAUDE.md` Rule #3

4.3 Category 2: Testbench Architecture

4.3.1 2.1 TB Location Requirements (MANDATORY)

Requirement: Project-specific TB classes MUST be in project area, NOT framework

Directory Structure:

```
CORRECT:
projects/components/{name}/dv/tbclasses/{module}_tb.py

WRONG:
bin/TBClasses/{name}/{module}_tb.py
```

Import Pattern:

```
# Add repo root to path
import os, sys
repo_root = os.path.abspath(os.path.join(os.path.dirname(__file__),
'../../../../../..'))
sys.path.insert(0, repo_root)

# Import from PROJECT AREA
from projects.components.dma_ip.rapids.dv.tbclasses.scheduler_tb
import SchedulerTB

# Shared framework utilities
from TBClasses.shared.tbbase import TBBase
from CocoTBFramework.components.axi4.axi4_master import AXI4Master
```

Applies to: ALL project-specific testbenches **Source:** Root `/CLAUDE.md`, `projects/components/CLAUDE.md` Rule #4, `rapids/CLAUDE.md` Rule #0.1, `stream/CLAUDE.md` Rule #0.1, `amba/CLAUDE.md` Rule #0

4.3.2 2.2 Three Mandatory TB Methods (MANDATORY)

Requirement: EVERY testbench class MUST implement these three methods

Pattern:

```
class MyModuleTB(TBBase):
    async def setup_clocks_and_reset(self):
        """Complete initialization - starts clocks and performs
reset"""
        await self.start_clock('clk', freq=10, units='ns')

        # Set config before reset (if needed)
        self.dut.cfg_param.value = initial_value

        # Reset sequence
        await self.assert_reset()
        await self.wait_clocks('clk', 10)
        await self.deassert_reset()
        await self.wait_clocks('clk', 5)

    async def assert_reset(self):
        """Assert reset signal"""
        self.dut.rst_n.value = 0  # Active-low

    async def deassert_reset(self):
        """Deassert reset signal"""
        self.dut.rst_n.value = 1
```

Why Required: - Consistency across all testbenches - Reusability for mid-test resets - Clear test structure and intent - Configuration control (some RTL needs signals set BEFORE reset)

Applies to: ALL testbench classes **Source:** Root `/CLAUDE.md`, `rapids/CLAUDE.md` Rule #0.5, `stream/CLAUDE.md` Rule #0.2, `CocoTBFramework/CLAUDE.md` Rule #3

4.3.3 2.3 TBBase Inheritance (MANDATORY)

Requirement: ALL testbench classes MUST inherit from TBBase

Pattern:

```
from TBClasses.shared.tbbase import TBBase

class MyModuleTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)
        # Testbench-specific initialization
```

What TBBase Provides: - Clock management (`start_clock`, `wait_clocks`) - Reset utilities (`assert_reset`, `deassert_reset`) - Logging (`self.log`) - Progress tracking (`mark_progress`) - Common utilities (`convert_to_int`, `format_dec`)

Applies to: ALL testbench classes **Source:** `CocoTBFramework/CLAUDE.md` Rule #2

4.3.4 2.4 Three-Layer Architecture (MANDATORY)

Requirement: Verification MUST be separated into three layers

Layers:

1. **Testbench (TB):** `projects/components/{name}/dv/tbclasses/{module}_tb.py`
   - Infrastructure + BFMs
   - Clock/reset control
   - Base test methods
   - NO verification logic
2. **Scoreboard:** `bin/TBClasses/scoreboards/{protocol}/{module}_scoreboard.py`
   - Verification logic
   - Expected vs actual comparison
   - Coverage collection
3. **Test:** `projects/components/{name}/dv/tests/test_{module}.py`
   - Test intelligence
   - **Imports** TB class (does NOT define it)
   - Pytest wrappers
   - Scenario logic

NEVER Embed TB Classes in Test Files

Applies to: ALL verification code **Source:** `amba/CLAUDE.md` Rule #0, `rapids/CLAUDE.md` Rule #0.1, `CocoTBFramework/CLAUDE.md` Rule #4

4.3.5 2.5 Queue-Based Verification (PREFERRED)

Requirement: Use direct queue access for simple tests, NOT memory models

Pattern:

```
# CORRECT: Direct queue access
aw_pkt = self.aw_monitor._recvQ.popleft()
w_pkt = self.w_monitor._recvQ.popleft()

# Verify
assert aw_pkt.addr == expected_addr
assert w_pkt.data == expected_data

# WRONG: Memory model for simple test
memory_model = MemoryModel()
written_data = memory_model.read(addr, 4)  # Unnecessary complexity
```

When to Use: - **Queue Access:** Simple in-order tests, single-master, no overlap - **Memory Models:** Complex OOO scenarios, multi-master with address overlap

Applies to: Simple in-order verification scenarios **Source:** `rapids/CLAUDE.md` Scoreboard Pattern, `CocoTBFramework/CLAUDE.md` Rule #5

4.4 Category 3: Test Standards

4.4.1 3.1 Test Naming Convention (MANDATORY)

Requirement: Pytest function MUST match module name

Pattern:

```
# Module: axi4_dwidth_converter_wr.sv

CORRECT:
def test_axi4_dwidth_converter_wr(request, params):
    """Test for axi4_dwidth_converter_wr.sv"""
    ...

WRONG:
def test_converter(request, params):  # Too generic!
    ...
```

Why: Prevents pytest collection conflicts between related modules

Applies to: ALL pytest test functions **Source:** Root `/CLAUDE.md`

4.4.2 3.2 Test Pattern - CocoTB + Pytest (MANDATORY for projects/components/)

Requirement: Use two-tier pattern with cocotb_test_* prefix

Pattern:

```
# 1. CocoTB test functions - prefix with "cocotb_test_*"
@cocotb.test(timeout_time=100, timeout_unit="ms")
async def cocotb_test_basic(dut):  # ← cocotb_test_* prefix!
    """CocoTB test function - NOT collected by pytest"""
    tb = SimpleSRAMTB(dut)
    await tb.setup_clocks_and_reset()
    # ... test logic

# 2. Pytest wrapper - calls specific cocotb_test_* function
@pytest.mark.parametrize("addr_width, data_width", params)
def test_basic(request, addr_width, data_width):
    """Pytest wrapper - calls cocotb_test_basic"""
    run(
        # ... setup
        testcase="cocotb_test_basic",  # ← Explicitly specify
        # ... compilation args
    )
```

Why: - Pytest collects only wrappers, not CocoTB functions - Each wrapper calls ONE specific CocoTB function - Allows parameter sweeps at pytest level

Applies to: ALL tests in projects/components/ **Source:** Root `/CLAUDE.md`

4.4.3 3.3 100% Success Requirement (MANDATORY)

Requirement: ALL tests MUST achieve 100% success rate

Pattern:

```
# WRONG: Accepting partial success
assert success_rate >= 70  # "Good enough" - NO!

# CORRECT: Require 100% success
assert success_rate >= 100  # 100% required
```

Rationale: - RTL is deterministic - 100% success is achievable - Partial success indicates bugs or timing issues - Lower thresholds mask real problems - Tests must reliably catch regressions

Applies to: ALL tests **Source:** `rapids/CLAUDE.md`, `CocoTBFramework/CLAUDE.md`

4.4.4 3.4 Simulation Timestamps in Logs (MANDATORY)

Requirement: ALL test log messages MUST include simulation time

Pattern:

```
# CORRECT: Include simulation timestamp
self.log.info(f"@ {cocotb.utils.get_sim_time('ns')}ns: Beat {i}:
data=0x{data:X}, s_ready={ready}, occupancy={occ}")

# WRONG: No simulation timestamp
self.log.info(f"Beat {i}: data=0x{data:X}, s_ready={ready},
occupancy={occ}")
```

Output Example:

```
2025-11-06 19:11:20,688 - INFO - @ 150ns: Beat 1: data=0xA000,
s_ready=1, occupancy=0
2025-11-06 19:11:20,688 - INFO - @ 160ns: Beat 2: data=0xA001,
s_ready=1, occupancy=1
2025-11-06 19:11:20,688 - INFO - @ 170ns: Beat 3: data=0xA002,
s_ready=1, occupancy=2
2025-11-06 19:11:20,688 - INFO - @ 180ns: Beat 4 BLOCKED: s_ready=0,
occupancy=3
```

Why Critical: - Correlate test events with waveform timing - Debug timing-sensitive failures - Understand cycle-accurate behavior - Match log messages to specific clock edges

What Counts as "Simulation Time": - Use `cocotb.utils.get_sim_time('ns')` for nanoseconds - Use `cocotb.utils.get_sim_time('ps')` for picoseconds - Format: `@ {time} {unit}:` at START of message

Anti-Pattern:

```
# WRONG: Only Python logging timestamp
self.log.info(f"Beat {i} completed")
# Output: "2025-11-06 19:11:20,688 - INFO - Beat 1 completed"
# Problem: Can't correlate with waveform!
```

Applies to: ALL test log messages showing DUT behavior **Priority:** P0 - CRITICAL for debugging **Source:** User feedback 2025-11-06

4.5 Category 4: Framework Usage

4.5.1 4.1 Search Before Creating (MANDATORY)

Requirement: ALWAYS search existing modules/components BEFORE creating new

Search Commands:

```
# Search for existing RTL modules
find rtl/{subsystem}/ -name "*.sv" | xargs grep -l "keyword"

# Search for existing BFMs (CocoTBFramework, editable-installed from
RTLDesignSherpa-DV)
find "$(python3 -c 'import
CocoTBFramework,os;print(os.path.dirname(CocoTBFramework.__file__))')/
components" -name "*.py" | head

# Search for existing TB classes
find bin/TBClasses/ -name "*_tb.py"
```

Decision Tree: - Exact match exists → Use it - Close match exists → Adapt with parameters - No match found → Document search, propose new - Didn't search → STOP, go back and search

Applies to: RTL modules, BFMs, TB components **Source:** `rtl/common/CLAUDE.md` Rule #1, `CocoTBFramework/CLAUDE.md` Rule #1

4.5.2 4.2 BFM Usage for Protocols (MANDATORY for RAPIDS)

Requirement: MUST use framework BFMs for all protocol interfaces

BFM Selection: - **Custom valid/ready:** GAXI Master/Slave (`CocoTBFramework/components/gaxi/`) - **AXI4:** AXI4 Master/Slave (`CocoTBFramework/components/axi4/`) - **APB:** APB drivers (`CocoTBFramework/components/apb/`) - **AXI-Stream:** AXIS drivers (`CocoTBFramework/components/axis4/`)

(BFMs come from the RTLDesignSherpa-DV framework repo; shared TB classes in this repo are `bin/TBClasses/`)

NEVER: - Manually drive valid/ready handshakes - Create custom protocol drivers when framework has them

Applies to: All RAPIDS FUB-level tests **Source:** `rapids/CLAUDE.md` Rule #1

4.6 Category 5: Documentation

4.6.1 5.1 No Emojis in Technical Docs (MANDATORY)

Requirement: No emojis in technical specifications

Rationale: - Break PDF generation tools (LaTeX) - Appear unprofessional in formal documentation - Use plain text for all technical documentation

Exception: User explicitly requests emojis (rare)

Applies to: All technical documentation **Source:** Root `/CLAUDE.md`

4.7 Category 6: Project-Specific Requirements

4.7.1 6.1 Commit Messages Carry No Attribution Trailer

Requirement: A commit message is the change and nothing else. No `Co-Authored-By:`, no `Claude-Session:`, no "generated with" line, and no "Documentation and implementation support by Claude."

Applies to: every commit in this repo, every area, no exceptions. **Decided:** 2026-08-31 (owner). Supersedes the former per-component attribution formats that lived here as 6.1/6.2 and in six CLAUDE.md files; those required a trailer and contradicted this. Practice already matched this rule – the twenty commits before it was written carried zero trailers – but the rule itself was recorded nowhere in the repo, so every session rediscovered the conflict. It is written down here to stop that.

Not enforced by tooling. No commit-msg hook exists; `bin/hooks/pre-commit` checks task IDs, declaration order, protocol families, scenario verdicts, doc examples and .sv parse. This one is on the author.

Rationale and the history: `vault/handbook/authoring/commit-messages.md`.

4.7.2 6.2 STREAM Tutorial Focus

Requirement: STREAM has intentional simplifications - DO NOT "fix" them

Intentional Simplifications: 1. Aligned addresses only - No alignment fixup logic 2. Length in beats - Not bytes or chunks 3. No circular buffers - Explicit chain termination only 4. No credit management - Simple transaction limits 5. Pure memory-to-memory - No network interfaces

When users ask "Can we add alignment fixup?" - **Correct answer:** "STREAM intentionally keeps addresses aligned for tutorial simplicity. For complex alignment, see RAPIDS." - **Wrong answer:** "Sure, let me add alignment logic..." (defeats tutorial purpose!)

Applies to: STREAM component only **Source:** `stream/CLAUDE.md` Rule #0.1

4.8 Summary by Priority

4.8.1 P0 - Critical (Enforcement Required)

1. Reset macro usage (`projects/components/`)
2. TB location (project area, not framework)
3. Three mandatory TB methods
4. TBBase inheritance
5. Three-layer architecture
6. 100% test success requirement

4.8.2 P1 - High Priority (Strong Recommendation)

7. FPGA synthesis attributes
8. Array syntax standard
9. SRAM module standards
10. Test naming convention
11. CocoTB + Pytest pattern
12. Queue-based verification

4.8.3 P2 - Standard Practice (Best Practices)

13. Search before creating
14. BFM usage for protocols
15. Active-low reset convention
16. No emojis in technical docs

4.8.4 P3 - Project-Specific

17. No attribution trailer on commit messages
18. STREAM tutorial focus

4.9 Compliance Checklist

Use this checklist when creating new RTL or testbenches:

RTL Module: - [ ] Uses `ALWAYS_FF_RST` macro (projects/components/) - [ ] Memory arrays have FPGA attributes - [ ] Arrays use `[DEPTH]` syntax - [ ] SRAM modules have no reset port - [ ] Uses active-low reset (`rst_n` or `aresetn`)

Testbench Class: - [ ] Located in `projects/components/{name}/dv/tbclasses/` - [ ] Inherits from TBBase - [ ] Implements `setup_clocks_and_reset()` - [ ] Implements `assert_reset()` - [ ] Implements `deassert_reset()` - [ ] Uses framework BFMs (not manual handshaking)

Test File: - [ ] Imports TB class from project area - [ ] CocoTB functions prefixed with `cocotb_test_*` - [ ] Pytest function matches module name - [ ] Uses `testcase=` to specify CocoTB function - [ ] Requires 100% success rate - [ ] Uses queue-based verification (for simple tests)

Documentation: - [ ] No emojis in technical specs - [ ] Commit message carries no attribution trailer

---

4.10 Enforcement

These requirements are enforced via:

1. **Code Review** - All PRs reviewed for compliance
2. **Automated Tools:**
   - `bin/update_resets.py` - Reset macro conversion
   - `bin/audit_signal_naming_conflicts.py` - Signal naming checks
3. **CI/CD Checks:**
   - Verilator linting
   - Pytest test execution
   - Test success rate validation
4. **Documentation:**
   - CLAUDE.md files per subsystem
   - PRD.md requirements per component

Non-Compliance: - PRs will be REJECTED if they violate P0 requirements - P1 violations require justification and approval - P2/P3 violations should be corrected but may not block PR

4.11 References

Main Documentation: - `/CLAUDE.md` - Repository-wide guidance - `/PRD.md` - Master requirements - `projects/components/CLAUDE.md` - Project area standards - `CLAUDE.md` in the RTLDesignSherpa-DV repo - DV framework patterns (shared TB classes: `bin/TBClasses/`)

Subsystem Documentation: - `projects/components/dma-ip/rapids/CLAUDE.md` - RAPIDS guidance - `projects/components/dma-ip/stream/CLAUDE.md` - STREAM guidance - `rtl/amba/CLAUDE.md` - AMBA subsystem guidance - `rtl/common/CLAUDE.md` - Common RTL library guidance

Tools: - `bin/update_resets.py` - Reset macro conversion - `bin/audit_signal_naming_conflicts.py` - Signal naming audit - `bin/md_to_docx.py` - Documentation generation

---

Version: 1.0 **Last Updated:** 2025-10-31 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Document Information

Table 0.1: Document information

Field	Value
Title	amber — Blocking MESI Snoopy L1 Cache — Hardware Architecture Specification
Version	0.6
Date	2026-10-11
Status	HAS ratifies the Pre-HAS sketch; book-informed pass folded Chinnakonda multiprocessor-cache material into coherence FSM, snoop adapter, and future hooks as candidates/hooks. PRD decisions D1, D2, D4, D5, D6, D7, D8, D9, D10 are DECIDED. D3 (memory-side shape) and D11 (array construction) remain OPEN and are recorded in the chapters they shape.
Scope	`amber` and `amber_ace` tops, the shared `amber_core`, interfaces, parameters, and integration rules for the pair rig and the onyx rig.
Not in scope	Lockup-free miss handling (jet), directory-based coherence, L2/L3, ECC, TLBs, multi-port CPU issue, ACE DVM and barrier semantics.

5.1 Related documents

Table 0.2: Related documents

Document	Where	Relationship
amber PRD	../../../PRD.md	**binding requirements** — all decisions and success criteria originate here

Document	Where	Relationship
amber Pre-HAS	../amber_prehas.md	architecture sketch ratified by this book
jet PRD	../../../../jet-mesi-l1/PRD.md	lockup-free follow-on; amber must not pre-decide its open rows
onyx PRD	../../../../onyx-ace-ccu/PRD.md	defines the ACE-shaped snoop port (D7) and the coherent-transaction subset (D2)
AMBA ACE Interface Definition	../../../../References/AMBA_ACE_Interface_Definition.md	in-repo ACE contract, verified against IHI 0022H
gem5 Ruby protocols	../../../../References/gem5-ruby-protocols/README.md	source for the Python reference model and FSM oracles
cache simulator	../../../../../../../bin/apps/cache_sim/README.md	golden parity model for LRU/FIFO/RANDOM
Global Requirements	../../../../../../../GLOBAL_REQUIREMENTS.md	reset macros, array syntax, no bespoke SRAM, filelist registry
amber/jet improvement plan memo	../../amber-jet-improvement-plan.md	concurrent work; may 404 until the memo lands
Designing a Multiprocessor Cache Subsystem	external: `M. Chinnakonda`	book-informed candidates; sections cited in the chapters that use them

5.2 Terminology

Pair rig. Two `amber` instances plus shared memory; snoops travel peer-to-peer on the ACE-shaped port and misses are plain AXI4 to memory (amber D10).

Onyx rig. An `amber_ace` instance attached to the `onyx` coherency manager; misses become the onyx D2 coherent-transaction subset and snoops arrive from onyx.

Rig. The top-level instantiation you choose: `amber` or `amber_ace`. There is no `RIG` elaboration parameter; the rig is the top (Pre-HAS Q1, resolved 2026-10-06).

F-item. Feature sketch from the Pre-HAS (F1–F18); this HAS ratifies each decided sketch and records where it remains open.

5.3 Revision history

Table 0.3: Revision history

Version	Date	Change
0.5	2026-10-06	Full HAS. Ratifies Pre-HAS F1–F18 against PRD v0.5 decisions, fixes the two-top structure, the module set, the interface pin contracts, the parameter sketch, the verification strategy, and the open D3/D11 questions.
0.6	2026-10-11	Book-informed pass: Chinnakonda multiprocessor-cache material folded into coherence FSM, snoop adapter, and future hooks as candidates/hooks; nothing newly decided.

5.4 How to read this book

Chapter 1 states purpose and vocabulary. Chapter 2 is the system shape: use cases, features, context, and one block diagram. Chapter 3 is the architecture — the shared core, data flow, the blocking control FSM, the coherence FSM, and arbitration. Chapter 4 is the pin-level interface inventory. Chapter 5 is the parameter package. Chapter 6 is integration: clock/reset, verification, characterization, and future hooks.

6 Purpose and Scope

6.1 Purpose

amber is a parameterized, blocking, snoopy MESI L1 data cache for the Sherpa research and teaching flow. It is deliberately the simplest correct cache: one outstanding transaction, no MSHRs, no hit-under-miss, no merge logic. That deletion makes it a clean baseline for the follow-on `jet` lockup-free cache and a tractable target for SymbiYosys proofs and cache-simulator parity.

This document specifies amber's hardware architecture. It is written after the PRD decisions that fix geometry, write policy, MESI variant, replacement policy, CPU-side interface, snoop transport, observation, verification strategy, and first consumer, but while D3 (memory-side shape) and D11 (array construction) remain open. The open rows are recorded honestly with their current direction and consequences; nothing here is presented as decided that is not.

6.2 Scope

In scope:

- The two top-level wrappers, `amber` (pair rig) and `amber_ace` (onyx rig), and the shared `amber_core`.
- CPU-side GAXI slave interface (`amber_cpu_frontend`).
- Fabric-side plain AXI4 masters under `amber` and ACE masters plus coherent-issue logic under `amber_ace`.
- ACE-shaped snoop responder port (`amber_snoop_resp`) toward a peer cache or onyx.
- Blocking control FSM, pending-fill bypass, victim buffer, fill/drain engines, tag/data arrays, replacement policy engine, and observation wrapper.
- Parameterization, clocking/reset, verification strategy, characterization plan, and future hooks.

Out of scope (named as decisions, not omissions): lockup-free miss handling (jet), directory-based or L2/L3 coherence, ECC on arrays, virtual memory and TLB interaction, multi-port CPU issue, ACE DVM and full barrier semantics, speculative prefetching, and adaptive replacement.

6.3 Why a blocking cache matters

Blocking simplifies coherence because there is only one transaction in flight. A probe against a pending fill can be answered from a single bypass register (F9); there is no MSHR hazard matrix, no merge CAM, no response reorder buffer. The research value is the measured delta to `jet`: with arrays, policy, observation, and the MESI protocol held constant, the difference between the two IPs is strictly the miss-handling concurrency. amber makes that delta isolable.

7 Document Conventions

7.1 Signal naming

- `fub_*` — the flat upstream-facing side of a house AXI/ACE wrapper or adapter (e.g., `fub_axi_araddr`, `fub_acaddr`).
- `m_*` — the master-facing side toward the fabric or peer (e.g., `m_axi_araddr`, `m_axi_acaddr`).
- `s_*` / `i_*` / `o_*` — used for local module ports where the house convention prefers it.
- Active-low resets are named `rst_n` inside project components and `aresetn` on AMBA-facing pins, per the repo's reset convention.

7.2 Numbering and bases

- Addresses and data widths are decimal unless prefixed with `0x`.
- Bus bit ranges use Verilog-style `[MSB:LSB]`.
- Array dimensions use the house `[DEPTH]` unpacked syntax, not `[0:DEPTH-1]`.

7.3 Citation discipline

- PRD rows are cited as `amber Dn` (e.g., `amber D1`).
- Pre-HAS feature sketches are cited as `Fn` (e.g., `F9`).
- `onyx` rows are cited as `onyx Dn`.
- `jet` rows are cited as `jet Jn`.
- `OPEN` decisions are stated as open, with the current direction and the consequence for the architecture. No `TBD` or placeholder language is used.

7.4 Diagram conventions

- Block diagrams show data flow and module ownership; they are not cycle-accurate timing diagrams.
- Mermaid source is inlined in the chapter and also stored under `assets/mermaid/`; the trailing source line in each chapter gives the asset path.

7.5 Voice

This specification uses first-person-plural engineering voice: “amber does X because Y.” Every architectural claim is traceable to a PRD row, an F-item, a GLOBAL_REQUIREMENTS rule, or a real RTL port list.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

8 Definitions and Acronyms

Table 1.0: Definitions and acronyms

Term	Definition
ACE	AXI Coherency Extensions (Arm IHI 0022); the protocol shape used for the snoop port and, in the onyx rig, for coherent fabric transactions.
AC / CR / CD	ACE snoop channels: Address, Response, Data.
ARSNOOP / AWSNOOP	ACE transaction-type fields on read and write address channels.
CRRESP	Five-bit snoop response: `DataTransfer`, `Error`, `PassDirty`, `IsShared`, `WasUnique`.
MESI	Cache-coherence states: Modified, Exclusive, Shared, Invalid.
MOESI	MESI plus Owned state; amber reserves encoding headroom but does not implement O in v1.0.
Line	The cache allocation unit, parameterized but defaulting to 64 bytes.
Set	The group of ways indexed by the set portion of the address.
Way	One associative entry within a set.
Blocking	The cache accepts no new CPU request while a miss is outstanding.
MSHR	Miss Status Holding Register; amber deliberately has none.
Fill	The memory/peer response that writes a missing line into the data array.
Drain	The write-back of a dirty victim line to memory or the CCU.

Term	Definition
Pending-fill bypass	The single register that lets a snoop probe observe the post-fill state while the fill is still in flight (F9).
Victim buffer	A small staging buffer for a dirty line waiting to drain; amber uses depth 1.
GAXI	The house generic AXI-like valid/ready streaming bus used for the CPU-side port.
MonBus	The 128-bit observation packet bus emitted by `*_monlite` wrappers.
Pair rig	Two `amber` instances plus shared memory, the gated deliverable (D10).
Onyx rig	An `amber_ace` instance attached to the `onyx` coherency manager.
Pattern-B	The repo's cocotb GATE/FUNC/FULL verification grid naming.

9 Use Cases

9.1 Primary use case: a correct, observable, provable L1 cache baseline

amber exists so the repo has a first cache IP whose correctness can be checked end-to-end: against a software golden model, against formal proofs, and against hardware cost measurements. The blocking design keeps the coherence protocol in the foreground and the miss-handling mechanics in the background, which is exactly the research posture.

9.2 Secondary use cases

1. **Pair-rig coherence research.** Two `amber` instances plus shared memory form the smallest coherent system in which snoop responses, dirty write-backs, and exclusive-to-shared transitions can be observed and measured (D10).
2. **Onyx-rig CCU attachment.** `amber_ace` attaches to the `onyx` coherency manager and exercises the onyx D2 coherent-transaction subset, so the manager side can be built and verified against a real cache (F12).
3. **STREAM attachment.** The GAXI CPU-side port lets STREAM attach without an adapter; this is the documented future integration hook (D2).
4. **Teaching and trace parity.** The cache_sim trace-replay cross-check makes amber a runnable illustration of how geometry and replacement policy combine into hit/miss behavior.

9.3 What amber is not designed for

High-throughput single-core execution, multiprocessor systems larger than the pair rig, or any workload that depends on hit-under-miss concurrency. Those belong to `jet` or to future IPs.

10 Key Features

Table 2.0: Key features and their decisions

Feature	What amber does	PRD / F anchor
Blocking single-outstanding pipeline	One miss at a time; the front end freezes until fill + replay complete.	F1
Parameterized geometry	4–32 KiB capacity, 32–64 B lines, 2–8 ways; default 32 KiB / 64 B / 4-way (128 sets); tiny 16-set / 2-way config reserved for proofs.	D1
GAXI CPU-side slave	House valid/ready streaming port with skid/FIFO plumbing; STREAM attaches without an adapter.	D2
Write-back + write-allocate	Default policy; write-through/no-allocate retained as a bring-up mode only.	D5
Plain MESI on a 3-bit state field	Stable M/E/S/I; headroom for MOESI O-state as a later elaboration upgrade.	D6
Pluggable replacement	LRU (default), tree-PLRU, FIFO, RANDOM; LRU/FIFO/RANDOM have cache_sim golden parity; tree-PLRU is the timing-tight fallback without a sim golden model.	D7
ACE-shaped snoop responder	AC/CR/CD adapter facing peer or onyx; bus-	D4

Feature	What amber does	PRD / F anchor
	agnostic inside.	
Pending-fill bypass register	Probe-during-fill answered from the register, bounded and state-accurate.	F9
Two tops on one core	`amber` for the pair rig, `amber_ace` for the onyx rig; no mode parameter.	F12
MonBus observation	`amber_monlite` emits 128-bit packets; drop-and-count, never stalls the observed path.	D8

The feature set is intentionally narrow. Every item that is not listed is a non-goal: no MSHRs, no prefetch, no ECC, no multi-port issue, no directory filter, no barrier/DVM support. That narrowness is the design.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 System Context

11.1 Where amber sits

amber is a cache IP on the AMBA fabric. It consumes GAXI transactions from a CPU or DMA master on one side and issues AXI4 or ACE transactions toward memory or a coherency manager on the other. A third ACE-shaped port receives snoops from a peer cache or from `onyx`. Observation leaves through a fourth MonBus tap.

11.2 Pair rig (`amber` top)

```
CPU/STREAM --GAXI--> amber --AXI4--> memory
                         ^--AC/CR/CD-- peer amber
```

The pair rig is the gated deliverable (D10). Each `amber` has its own tag/data arrays and its own replacement state. Coherence is snoopy and peer-to-peer: when one cache misses on a line the other holds, the response is delivered through the ACE-shaped snoop port, not through memory. Dirty victims write back to memory over the plain AXI4 master.

11.3 Onyx rig (`amber_ace` top)

```
CPU/STREAM --GAXI--> amber_ace --ACE--> onyx --AXI4--> memory
                                        ^--AC/CR/CD--
```

In the onyx rig, `amber_ace` behaves as an ACE master toward `onyx` and a snoop responder toward `onyx`. Coherent read misses become `ReadShared` or `ReadUnique`; write promotions become `CleanUnique` or `MakeUnique`; dirty evictions become `WriteBack`; clean evictions become `Evict` (onyx D2 subset). Memory is on the far side of `onyx`, not directly reachable by `amber_ace`.

11.4 Shared core, different tops

Both rigs instantiate the same `amber_core`. The CPU-side front end, the tag/data arrays, the control FSM, the replacement engine, and the snoop responder are identical. Only the fabric master stack and the coherent-transaction issuer differ, and that difference lives in the top wrapper, not in the core.

12 Block Diagram

Figure 2.1 shows the shared `amber_core` and the two rig tops that instantiate it. The core is rig-agnostic: the CPU-side front end, control FSM, arrays, replacement engine, fill/drain engines, snoop responder, and observation wrapper are identical in both configurations. The pair-rig top adds plain AXI4 masters to memory; the onyx-rig top adds ACE masters and the `amber_ace_issue` coherent-transaction issuer.

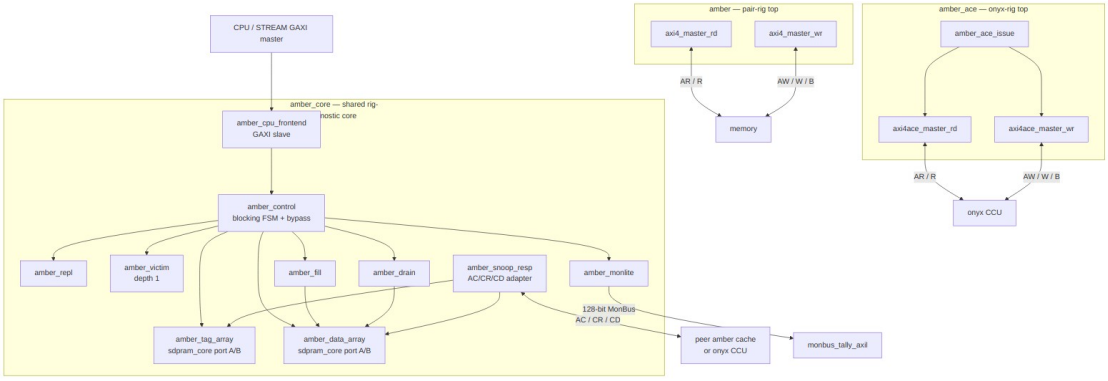


Diagram 1

Source: 01_block_diagram.mmd — the fence above mirrors it.

12.1 Module roles

Table 2.1: Module roles

Module	Role
`amber`	Pair-rig top: `amber_core` + plain AXI4 masters + snoop responder toward peer.
`amber_ace`	Onyx-rig top: `amber_core` + ACE masters + `amber_ace_issue` + snoop responder toward onyx.
`amber_core`	Shared rig-agnostic core instantiated by both tops.
`amber_cpu_frontend`	GAXI slave: request acceptance, response return, replay latch.
`amber_control`	Blocking pipeline FSM; owns tag-array control arbitration and the pending-fill bypass register.

Module	Role
amber_tag_array / amber_data_array	Dual-port sdpram_core stores; port A CPU/fill, port B snoop.
amber_repl	Pluggable replacement policy engine.
amber_victim	Dirty victim buffer + drain trigger; depth 1.
amber_fill / amber_drain	Read/write engines; plain AXI4 under amber, ACE under amber_ace.
amber_snoop_resp	ACE AC/CR/CD adapter; bus-agnostic inside.
amber_ace_issue	Maps cache events to the onyx D2 coherent-transaction subset.
amber_monlite	Observation wrapper; emits MonBus packets, never stalls.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 Top Structure and the Two Rigs

13.1 No rig parameter

amber does not use a `RIG` elaboration parameter. The rig is the top you instantiate: instantiate `amber` for the pair rig, instantiate `amber_ace` for the onyx rig (F12). Both tops compile; both are exercised in the DV matrix. This keeps the rig-specific wiring out of the shared core and avoids a mode signal that would broaden the formal surface for no benefit.

13.2 `amber` — pair-rig top

`amber` wraps `amber_core` with:

- House `axi4_master_rd` and `axi4_master_wr` for plain AXI4 memory access (D3 direction).
- The `amber_snoop_resp` ACE adapter facing the peer cache.
- No coherent-transaction issuer; memory-side misses are ordinary AR/AW bursts.

The snoop responder sees the peer's AC/CR/CD even though the memory side is plain AXI4. That is exactly the pair-rig shape: coherence is handled cache-to-cache; memory only supplies cold misses and absorbs dirty write-backs.

13.3 `amber_ace` — onyx-rig top

`amber_ace` wraps the same `amber_core` with:

- `axi4ace_master_rd` and `axi4ace_master_wr` toward `onyx`.
- `amber_ace_issue`, which maps cache-side events to the onyx D2 subset: read miss → `ReadShared`/`ReadUnique`, write promotion → `CleanUnique`/`MakeUnique`, dirty eviction → `WriteBack`, clean eviction → `Evict`.
- The same `amber_snoop_resp` adapter, but facing `onyx` rather than a peer.

Memory is not directly reachable; all fabric traffic is coherent and routed through `onyx`.

13.4 What is inside `amber_core`

`amber_core` contains everything that does not change between rigs: `amber_cpu_frontend`, `amber_control`, `amber_tag_array`, `amber_data_array`, `amber_repl`, `amber_victim`, `amber_fill`, `amber_drain`, `amber_snoop_resp`, and `amber_monlite`. The fill/drain modules

expose a rig-agnostic internal interface; the top selects the AXI4 or ACE transport adapter that drives it.

---

14 Data Flow: Hits, Misses, Fills, and Drains

14.1 Hit path

A CPU request accepted by `amber_cpu_frontend` is latched and presented to `amber_control`. The control FSM drives a tag lookup on port A of `amber_tag_array`. On a hit, the state field is checked: read returns data from `amber_data_array` port A; write updates data and promotes the line to Modified. The response returns through the front end in the same transaction.

14.2 Miss path

On a miss, `amber_repl` selects the victim way. If the victim is dirty, it is moved to `amber_victim` and `amber_drain` issues a write-back burst. Once the victim path is clear, `amber_fill` issues a read burst. Returned beats write `amber_data_array` port A; the tag and MESI state are installed; the original request replays from the front-end latch. Figure 3.1 shows the sequence.

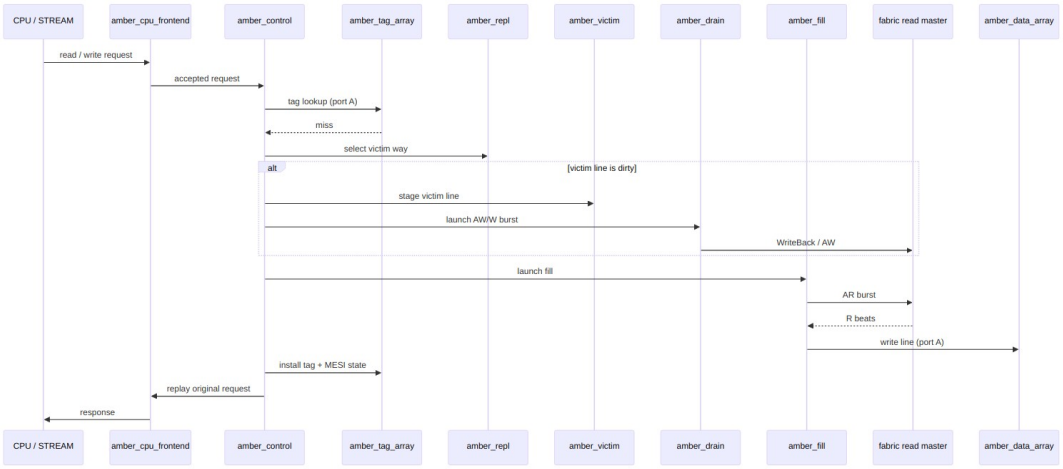


Diagram 2

Source: 02_miss_flow.mmd — the fence above mirrors it.

14.3 Snoop path

A snoop arrives at `amber_snoop_resp`, is buffered, and is presented to `amber_control`. The control FSM reads `amber_tag_array` port B. If the address matches the pending-fill bypass register, the response is composed from the register's post-fill state and any fill beats already received. Otherwise the tag state determines CRRESP and whether `amber_data_array` port B supplies CD beats. The adapter returns CR and CD in order to the requester.

14.4 Open shape: memory-side masters (D3)

D3 is open at the time of this writing. The direction is plain AXI4 read/write masters on the house `axi4_master_rd/wr` wrappers for the pair rig, and the existing `axi4ace_master_rd/wr` for the onyx rig. The exact skid depths, ID width, and user-width defaults follow the wrappers' own parameters; the cache does not override them except where geometry or bus width requires it. Finalizing D3 means pinning those defaults and the AW/AR burst length calculation in the fill/drain modules.

14.5 Open shape: array construction (D11)

D11 is open: tag and data stores will be built from the shared `sdpram_core` primitive, with FPGA synthesis attributes per `GLOBAL_REQUIREMENTS.md` and the `[DEPTH]` array syntax. The dual-port behavior (port A CPU/fill, port B snoop) is fixed; the exact banking, write-masking, and byte-enable plumbing are micro-architecture details for the MAS.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 Control FSM: Blocking Pipeline and Pending-Fill Bypass

15.1 The blocking pipeline

`amber_control` implements the cache's only state machine. It moves through a fixed sequence per request:

1. **Lookup** — read tag and state for the request's set on port A.
2. **Hit service** — read or write the data array and return the response.
3. **Miss resolution** — on a miss, ask `amber_repl` for a victim way.
4. **Victim drain** — if the victim is dirty, stage it in `amber_victim` and hand it to `amber_drain`.
5. **Fill request** — launch `amber_fill` for a whole-line burst.
6. **Fill completion** — write data and install tag + MESI state.
7. **Replay** — re-present the original request from the front-end latch; it now hits.

Because the pipeline is blocking, no new CPU request is accepted while a miss is in flight. That single-outstanding property is what makes the pending-fill bypass tractable.

15.2 Pending-fill bypass register

While a fill is outstanding, a snoop may arrive for the same line. `amber_control` keeps one pending-fill register holding `{line_address, state_being_installed, data_available_mask}`. A snoop matching the pending line is answered from the register:

- `CRRESP` reflects the post-fill state (e.g., Shared for a read-shared fill, Exclusive for a read-unique fill, Invalid for a line that will be invalidated by the requester's MakeUnique).
- Required CD beats are forwarded from the fill stream as the beats arrive, so the response is bounded both for hits and for misses.

The correctness target is *state-accuracy*: a probe against a pending fill must observe exactly the post-fill state, never pre-fill. This is one of the SymbiYosys proof targets (D9).

15.3 FSM oracle derivation

The control FSM's stable and transient states are derived from the gem5 Ruby `MESI_Two_Level` SLICC tables, specifically `L1cache.sm` (D9). The Python reference model and the formal oracles

replay the same state transitions, so the protocol is cross-checked against an executable spec rather than prose. MOESI headroom is reserved in the encoding but the FSM v1.0 implements only M/E/S/I.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 Coherence FSM and the Snoop Responder

16.1 MESI state handling

Each tag entry carries a 3-bit state field. Only M/E/S/I are used in v1.0; the extra bit is reserved for MOESI O-state as a later elaboration upgrade (D6). State transitions follow the gem5 Ruby `MESI_Two_Level` SLICC tables and are cross-checked by the Python reference model and formal oracles (D9).

Table 3.0: MESI snoop CRRESP matrix — DT=DataTransfer, PD=PassDirty, IS=IsShared, WU=WasUnique. Snoop encodings are the six IHI0022 AC types; CleanUnique and MakeUnique are transactions a master issues (the onyx D2 subset), not snoop encodings, so they do not appear here.

Current state	Snoop type	Action	Next state	CRRESP (DT/PD/IS/WU/Err)
Modified	ReadShared	Transfer dirty data, pass dirty, downgrade to Shared	Shared	1 / 1 / 1 / 0 / 0
Modified	ReadOnce	Transfer dirty data, pass dirty, invalidate	Invalid	1 / 1 / 0 / 0 / 0
Modified	ReadUnique	Transfer dirty data, pass dirty, invalidate	Invalid	1 / 1 / 0 / 0 / 0
Modified	CleanShared	Transfer dirty data, pass dirty, downgrade to Shared	Shared	1 / 1 / 1 / 0 / 0
Modified	CleanInvalid	Transfer dirty data, pass dirty,	Invalid	1 / 1 / 1 / 0 / 0

Current state	Snoop type	Action	Next state	CRRESP (DT/PD/IS/WU/Err)
		invalidate		
Modified	MakeInvalid	Invalidate without data transfer (IHI0022 forbids DT here)	Invalid	0 / 0 / 0 / 0 / 0
Exclusive	ReadShared	Transfer clean data, downgrade to Shared	Shared	1 / 0 / 1 / 1 / 0
Exclusive	ReadOnce	Transfer clean data, downgrade to Shared	Shared	1 / 0 / 1 / 1 / 0
Exclusive	ReadUnique	Transfer clean data, invalidate	Invalid	1 / 0 / 0 / 1 / 0
Exclusive	CleanShared	Already clean; no transfer, no change	Exclusive	0 / 0 / 1 / 1 / 0
Exclusive	CleanInvalid	No data; invalidate	Invalid	0 / 0 / 0 / 0 / 0
Exclusive	MakeInvalid	No data; invalidate	Invalid	0 / 0 / 0 / 0 / 0
Shared	ReadShared	No data; keep Shared	Shared	0 / 0 / 1 / 0 / 0
Shared	ReadOnce	No data; keep Shared	Shared	0 / 0 / 1 / 0 / 0
Shared	ReadUnique	No data; invalidate	Invalid	0 / 0 / 0 / 0 / 0
Shared	CleanShared	No data; keep	Shared	0 / 0 / 0 / 0 / 0

Current state	Snoop type	Action	Next state	CRRESP (DT/PD/IS/WU/Err)
		Shared		
Shared	CleanInvalid	No data; invalidate	Invalid	0 / 0 / 0 / 0 / 0
Shared	MakeInvalid	No data; invalidate	Invalid	0 / 0 / 0 / 0 / 0
Invalid	any	No data; miss	Invalid	0 / 0 / 0 / 0 / 0

This matrix is the amber MESI interpretation of the cocotb-framework 1.2.0 ACE BFM default handler, bit for bit — including the release-corrected `MakeInvalid` behavior on Modified/Owned lines (invalidate, no data transfer, IHI0022 forbids DataTransfer there). Where a textbook MESI next-state could be argued either way (e.g. `Exclusive` + CleanShared: already clean, so the line stays `Exclusive` while the response carries `IsShared` per the family matrix), amber follows the BFM response bits and the gem5 `MESI_Two_Level` transient-state tables for the transitions. The DV scoreboard checks every CRRESP against this table, and `CRRESP.validate_for_snoop` is the in-BFM guard for the same rules.

16.2 Snoop flow

Figure 3.2 shows the probe path. The ACE adapter decouples ACE signaling from the cache core: inside `amber_core` the snoop is a bus-agnostic `{address, type, response, data}` transaction; only `amber_snoop_resp` speaks ACE.

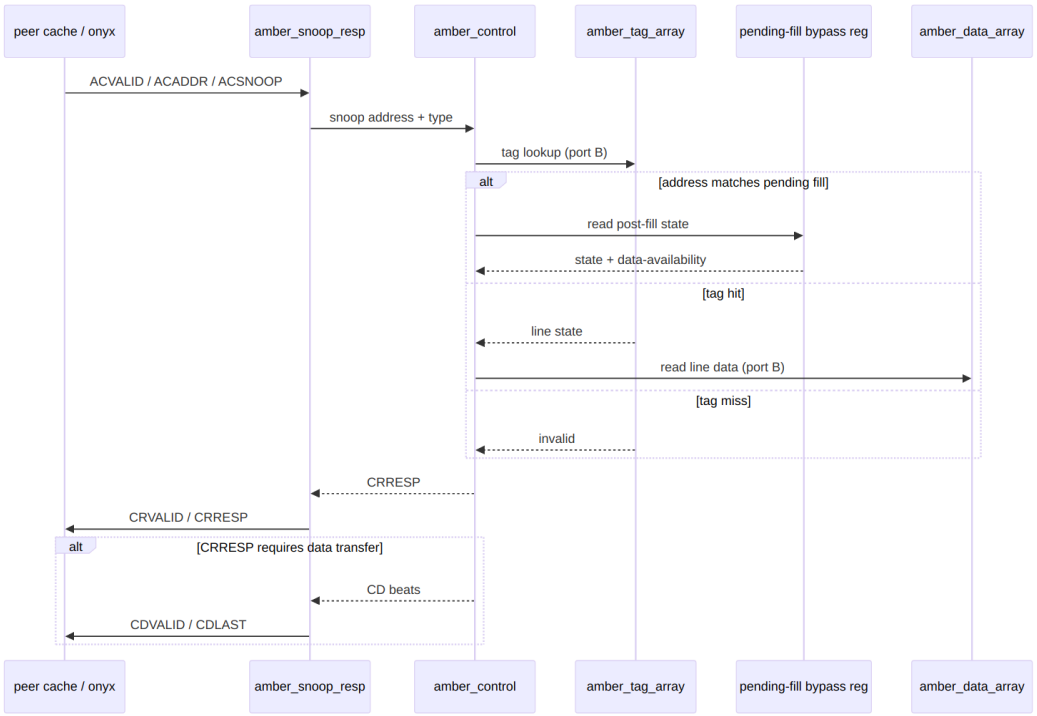


Diagram 3

Source: 03_snoop_flow.mmd — the fence above mirrors it.

16.3 Ordering convention

amber's adapter completes a transaction's CD beats before asserting its CR. That is not amber being stricter than the protocol — IHI0022 requires the snoop response only after the final CD beat, and the cocotb-framework 1.2.0 compliance checker (`check_cr_order` / `check_cd_order`) enforces exactly this on every gather. The convention matches the family decision recorded with onyx D4 and keeps the gather logic simple to verify.

16.4 Book-informed snoop behavior

Snoop-response sourcing (book-informed candidate). When a snoop address matches a pending fill, the coherence FSM must respond from the fill if the fill data has arrived, and must respond not-present if the data has not yet arrived. The snoop is never delayed waiting for unrelated resources such as victim buffers or write buffers; making the snoop response dependent on an event that may never occur breaks forward progress and can deadlock the system (Chinnakonda, §7.7, §8.2). amber's pending-fill bypass already carries state + data-availability; this rule sharpens how the bypass is used and is recorded as a candidate for the formal target list.

Temporary snoop-stall for non-speculative stores (future hook). Once the core holds ownership for a non-speculative store, incoming snoops may be held off only temporarily and are guaranteed to complete afterward. This is the classic store-ownership / store-exclusive safeguard: the block is permissible because the store completion is guaranteed, but the block must not be open-ended or dependent on an unpredictable event (Chinnakonda, §8.1). amber v1.0 does not implement exclusive stores, so this is recorded as a coherence-FSM hook for the future exclusive-support work; it does not change current snoop behavior.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 Arbitration and the Victim Bypass

17.1 Dual-port array arbitration

`amber_tag_array` and `amber_data_array` are dual-port `sdpram_core` instances:

- **Port A** serves CPU lookups, fill writes, and replay reads.
- **Port B** serves snoop lookups and snoop data readout.

The arrays themselves need no arbitration in the common case because CPU and snoop use separate ports. The arbitration problem is at the control level: the miss FSM and the snoop FSM may both need to advance tag state or start a fill/drain in the same cycle. `amber_control` resolves this with snoop-priority round-robin: a snoop is never starved, and a CPU miss waits a bounded number of cycles.

17.2 Victim bypass

`amber_victim` is a depth-1 buffer. When a miss evicts a dirty line, the line is staged while the fill is outstanding. If a snoop arrives for that victim line before the drain completes, the buffer can supply the data without re-reading the array. This is the victim bypass. It is shallow by design: depth 1 matches the single-outstanding miss model and keeps the formal surface small.

17.3 Liveness target

Under fair arbitration, neither snoops nor CPU misses can deadlock. The SymbiYosys proof set (D9) includes a no-deadlock / no-livelock target scoped to `amber_control`, `amber_snoop_resp`, the pending-fill bypass, and the victim handoff, proven at the tiny 16-set / 2-way geometry.

18 CPU-Side GAXI Slave

18.1 Interface decision

The CPU-side port is a GAXI slave (amber D2). We chose GAXI over a plain valid/ready native port and over an AXI4 slave because the house BFM/monitor coverage and skid/FIFO plumbing already exist, and STREAM can attach later without an adapter.

18.2 Channel structure

The CPU-side port is **two house-GAXI streams** (amber D2) — `wr_valid`/`wr_ready`/`wr_data` in, `rd_valid`/`rd_ready`/`rd_data` out. The house GAXI pattern and its plumbing (`gaxi_skid_buffer`, `gaxi_fifo_sync`) are documented with the `rtl/amba/gaxi/` modules, not restated here. `amber_cpu_frontend` is the slave: it receives the request stream and drives the response stream.

Request stream (CPU → amber, slave input):

Signal	Direction	Width	Payload
`cpu_req_wr_valid`	in	1	request valid
`cpu_req_wr_ready`	out	1	request accepted
`cpu_req_wr_data`	in	`CPU_REQ_W`	packed {`addr`, `we`, `be`, `wdata`}

Response stream (amber → CPU, slave output):

Table 4.0: CPU-side GAXI slave channels

Signal	Direction	Width	Payload
`cpu_rsp_rd_valid`	out	1	response valid
`cpu_rsp_rd_ready`	in	1	response accepted
`cpu_rsp_rd_data`	out	`CPU_RSP_W`	packed {`rdata`}

Field packing follows the house field-width convention the AMBA wrappers use (`CPU_REQ_W = ADDR_WIDTH + 1 + BUS_WIDTH/8 + BUS_WIDTH`, `CPU_RSP_W = BUS_WIDTH`; the frontend's field config fixes the bit ranges). If the CPU agent prefers named fields at the boundary, the flattened per-field view (`fub_*`-style, the way the AMBA wrappers expose their FUB side) carries the same payload — the packed GAXI streams are the contract; naming is RTL bring-up detail.

The frontend latches the accepted request and holds it until `amber_control` returns the response. Because the cache is blocking, the response stream is naturally in-order and carries no ID tags. Write-allocate fills are read-modify-write sequences inside `amber_control`; the GAXI slave itself sees only single-beat reads and writes.

18.3 No register block

amber is a parameter-only IP. There is no APB CSR or register block. All visibility — configuration, event counting, and debug — comes through the MonBus observation fabric (`amber_monlite`). This is a deliberate decision, not an omission, and it keeps the CPU-side port the only software-visible surface.

19 Fabric-Side AXI4 Masters

19.1 Where they live

The `amber` top uses the house `axi4_master_rd` and `axi4_master_wr` wrappers for plain AXI4 memory access. These masters are functionally AXI4; they carry no ACE fields and no snoop channels. Fill requests become AR bursts; dirty victim drains become AW/W/B bursts.

19.2 Burst behavior

amber issues whole-line fills and whole-line drains. The burst length is `LINE_BYTES / (BUS_WIDTH/8)` beats. The address is aligned to the line size; `ARSIZE` and `AWSIZE` are derived from `BUS_WIDTH`; `ARBURST`/`AWBURST` are `INCR`. Single-outstanding operation means there is one AR transaction and one AW transaction in flight at any time, so no reorder buffer is needed.

19.3 Port summary

The plain AXI4 master stack exposes the standard five AXI4 channels toward memory:

Table 4.1: Plain AXI4 channels in the pair rig

Channel	Direction from amber	Purpose
AR	out	read address for fill
R	in	read data for fill
AW	out	write address for dirty victim drain
W	out	write data for dirty victim drain
B	in	write response for dirty victim drain

The exact pin names and widths follow the `axi4_master_rd` and `axi4_master_wr` module parameters. The cache supplies address, length, size, and burst type; ID and user fields pass through or are tied to defaults configurable at the top.

19.4 D3: decided

The memory-side interface is PRD **D3, decided 2026-10-07 (v0.6)**: AXI4 read/write masters on the house `axi4_master_rd`/`axi4_master_wr` wrappers (the stream/rapids pattern), observed through `*_monlite` per D8 — chosen over GAXI (D2 already spent that on the CPU side) and over a simple SRAM bring-up port. The fill/drain engines drive the `fub_axi_*` upstream side of those

wrappers; ID/user fields take the wrappers' parameter defaults; skid depths are the wrappers' house defaults. The pair rig's shared memory hangs off the `m_axi_*` sides through house fabric (Chapter 6).

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 Fabric-Side ACE Masters and Coherent Issue

20.1 Onyx-rig fabric

amber_ace uses the existing rtl/amba/ace/axi4ace_master_rd and axi4ace_master_wr modules toward onyx. These are full ACE masters with ARSNOOP[3:0] and AWSNOOP[2:0] transaction-type fields and auto-pulsed RACK/WACK outputs.

20.2 amber_ace_issue

amber_ace_issue maps cache-side events to the onyx D2 coherent-transaction subset:

Table 4.2: Cache events to ACE transactions

Cache event	ACE transaction	Snoop field
read miss, shared intent	ReadShared	fub_axi_arsnoop[3:0] = ReadShared encoding
read miss, exclusive intent	ReadUnique	fub_axi_arsnoop[3:0] = ReadUnique encoding
write to Shared line	CleanUnique	fub_axi_awsnoop[2:0] = CleanUnique encoding
write to Invalid line (whole line)	MakeUnique	fub_axi_awsnoop[2:0] = MakeUnique encoding
dirty eviction	WriteBack	fub_axi_awsnoop[2:0] = WriteBack encoding
clean eviction	Evict	fub_axi_awsnoop[2:0] = Evict encoding

20.3 House-module pin contracts — not restated here

axi4ace_master_rd and axi4ace_master_wr are house modules: their full fub_axi_*/m_axi_* pin contracts, widths, and parameters live in their own sources (rtl/amba/ace/axi4ace_master_rd.sv, rtl/amba/ace/axi4ace_master_wr.sv) and the AMBA documentation index — this HAS does not duplicate them. What amber adds on top of those contracts is exactly two facts:

- the fub_axi_arsnoop[3:0] / fub_axi_awsnoop[2:0] fields carry the Table 4.2 transaction encodings above (the standard ACE snoop values);

- `m_axi_rack` / `m_axi_wack` are **auto-pulsed inside the adapters** one cycle after the last R beat / the B handshake — the onyx D2 subset has no ordering semantics that need master-controlled acknowledges, so amber's `amber_ace_issue` never drives them.

20.4 Outside the onyx D2 subset

`WriteLineUnique`, `WriteUnique`, `WriteClean`, `WriteEvict`, and DVM/barrier transactions are outside the onyx D2 coherent-transaction subset that amber targets. The book treats these as scope-growing features: `WLU`/`WU` for non-temporal or partial-line writes (Chinnakonda, §7.3–7.4), `WriteClean`/`WriteEvict` for cache-maintenance and victim-cache traffic (Chinnakonda, §7.4), and address-based cache maintenance / DVM as a later coherent-system concern (Chinnakonda, §7.16). amber does not need them for v1.0; Table 4.2 is the supported subset.

21 Snoop Port: AC/CD/CR Adapter

21.1 Adapter purpose

`amber_snoop_resp` is the ACE-shaped snoop responder (amber D4). It decouples the cache core from ACE signaling: the fabric side speaks real ACE AC/CR/CD; the core side is a bus-agnostic `{address, type, response, data}` transaction.

21.2 House-module pin contract — not restated here

`axi4ace_snoop_slave` is a house module: its full `m_axi_*`/`fub_*` pin contract lives in `rtl/amba/ace/axi4ace_snoop_slave.sv` and the AMBA documentation index — this HAS does not duplicate it. The amber-visible facts are the boundary rule and the ordering guarantee in the next section; on the internal side, the cache core drives CRRESP combinationally at the `fub_acvalid/fub_acready` grant and sources CD beats on `fub_cdvalid/fub_cdready`, one transaction at a time (single control FSM).

21.3 Ordering and the family convention

Per the in-repo ACE definition, CR and CD must not be asserted before the AC handshake, and responses are returned in the order of the AC addresses. The book reinforces that ACE snoops carry no ID and the snoop channels are strictly in-order, so the adapter's CR+CD ordering obligation is absolute (Chinnakonda, §7.1, §14.18). amber strengthens this: CD beats complete before CR is asserted for the same transaction, giving onyx a single closed response at a time. Multiple outstanding snoops are buffered by the skid FIFOs inside the adapter; the cache core processes them one at a time because `amber_control` is a single FSM. The response bound is therefore one snoop at a time through the core; the adapter FIFOs absorb any back-pressure.

21.4 RACK/WACK question (open interface issue)

The house `axi4ace_snoop_slave` auto-pulses RACK/WACK one cycle after the last R beat and the B handshake. The book notes that RACK/WACK can be omitted when the data-return and snoop paths are ordered to the coherence point, because the signals only exist to prevent a snoop from overtaking read/write response data inside the cluster (Chinnakonda, §7.20, §14.39). Whether amber can rely on that omission depends on the onyx side of the boundary; it is recorded as an open interface question tied to onyx PRD row N1.

21.5 Bus-agnostic core boundary

The `fub_*` side is the only boundary the cache core sees. If `onyx` is later replaced by a real ACE interconnect, only `amber_snoop_resp` (and the top-level wiring) changes; `amber_core` does not. That was the reason D4 chose an ACE-shaped transport with a thin adapter.

22 Observation: MonBus through amber_monlite

22.1 Observation decision

amber observes through *_monlite wrappers only, not the heavyweight _mon (amber D8). The rationale is measurement integrity: a _mon wrapper that stalls the port when its tracking tables fill would corrupt the very miss-latency numbers the cache exists to produce. amber_monlite is drop-and-count: packets are emitted when possible, dropped and counted when not, and the observed path never stalls.

22.2 MonBus packet content

amber_monlite emits 128-bit MonBus packets with the standard UNIT/AGENT ids. Event classes:

Table 4.6: MonBus event classes

Event class	Payload highlights
hit (read/write)	address set, way, MESI state before/after
miss	address set, way, miss class (compulsory/capacity/conflict when the golden model supplies it)
snoop	snoop type, hit/miss, response class
eviction	clean/dirty, address set/way
MESI transition	old state, new state, cause
fill/drain start+end	address, direction, transaction type

monbus_tally_axil tallies these packets exactly like every other component in the repo. Event classification uses the same cache_sim per-access miss-class computation: compulsory, capacity, and conflict.

22.3 Observer cost

The cost of observation is measured in the characterization sweep (Chapter 6.3): LUT/FF/RAM and fmax are reported with amber_monlite present and with it removed, so the claim “the observer does not perturb what it measures” is quantified.

23 amber_pkg and Build-Time Geometry

23.1 Parameter philosophy

amber is a parameter-only IP. All geometry, policy, and bus-width choices are elaboration parameters; there is no runtime register block. This matches the decision to put all visibility on MonBus rather than on an APB CSR.

23.2 Core parameters

Table 5.0: Core elaboration parameters

Parameter	Default	Range / set	Meaning
SETS	128	16–512	number of sets
WAYS	4	2–8	associativity
LINE_BYTES	64	32–64	cache line size in bytes
BUS_WIDTH	64	32–128	CPU/fabric data width in bits
REPL_POLICY	"LRU"	lru, tree_plru, fifo, random	replacement policy
WRITE_POLICY	"wb_wa"	wb_wa, wt_na	write-back/write-allocate or write-through/no-allocate bring-up mode
VICTIM_DEPTH	1	1–2	dirty victim buffer depth

23.3 Derived quantities

- SET_INDEX_WIDTH = $clog2(SETS)
- LINE_OFFSET_WIDTH = $clog2(LINE_BYTES)
- TAG_WIDTH = ADDR_WIDTH - SET_INDEX_WIDTH - LINE_OFFSET_WIDTH
- State field adds 3 bits to the tag store (MESI now, MOESI headroom reserved).
- FILL_BEATS = LINE_BYTES / (BUS_WIDTH/8)

23.4 Package contents

`amber_pkg` holds the parameter defaults, derived localparams, MESI state encodings, the replacement-policy enum, the write-policy enum, and the transaction-type encodings used by `amber_ace_issue`. The FSM oracles and the Python reference model both derive from the same gem5 Ruby `MESI_Two_Level` SLICC tables, so the encoding in `amber_pkg` is the single source of truth for RTL and DV.

23.5 Rig is not a parameter

There is no `RIG` parameter. The pair rig and the onyx rig are two separate top modules sharing `amber_core` (Pre-HAS Q1, resolved 2026-10-06). Elaboration selects the rig by selecting the top file to compile and instantiate.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

24 Clocking and Reset

24.1 Clock domain

amber is a single-clock IP. The CPU-side GAXI port, the cache core, the fabric masters, and the snoop adapter all run on `aclk`. There are no CDC paths inside `amber_core`; the house AXI/ACE wrappers handle any external CDC if the fabric runs at a different frequency.

24.2 Reset

All flops use the `ALWAYS_FF_RST` macro with active-low asynchronous assertion, per `GLOBAL_REQUIREMENTS.md` Category 1.1. The reset signal inside project components is `rst_n`; AMBA-facing pins use `aresetn`. Memory arrays (`sdpram_core`-based tag/data stores) have no reset port; their contents are initialized by software or by the first misses after reset.

24.3 Reset sequence

After reset deassertion, `amber_control` starts in the idle state. The tag and data arrays contain undefined values until the first accesses fill them. The front end accepts no request until the control FSM is idle and ready. There is no explicit cache invalidation hardware; invalidation is the natural state of unaccessed lines because the MESI state field is not reset and the first access to any set treats a mismatching/invalid tag as a miss.

24.4 Clock gating

The ACE master and snoop adapter modules expose `busy` outputs. These can be used by the top-level clock-gating cell to shut off the fabric-side clock when the cache is quiescent. The core's `busy` signal is `amber_control`'s non-idle state OR the pending-fill bypass register valid bit OR any skid-buffer occupancy in the adapters.

25 Verification Strategy

25.1 Pattern-B cocotb grids

The primary verification layer is Pattern-B GATE/FUNC/FULL cocotb grids over the geometry × policy × rig matrix (amber D9). The ACE BFMs are the RDS-DV cocotb-framework 1.2.0 ACE extensions: `AXI4ACEMasterRead`, `AXI4ACEMasterWrite`, and `AXI4ACESnoopSlave/Master`.

25.2 Golden-model parity

For LRU, FIFO, and RANDOM, amber's hit/miss/miss-class counts must match the cache_sim trace replay at identical geometry and policy. tree-PLRU has no cache_sim golden model today; it is cross-checked by RTL self-checks and is recorded as a future sim extension (Chapter 6.4). Miss classes follow cache_sim: compulsory, capacity, and conflict.

25.3 Formal targets

SymbiYosys is scoped to the control layer and proven at the tiny 16-set / 2-way geometry:

- No stale data served after an external write (pending-fill bypass correctness).
- No deadlock or livelock under fair arbitration.
- Victim-buffer handoff correctness.
- `amber_snoop_resp` AC/CR/CD ordering.

The proof surface is intentionally narrow: proving the whole cache data path at default geometry is not the goal; proving the control and snoop-responder logic that correctness depends on is.

25.4 Python reference model

The Python reference model and the FSM oracles derive from the gem5 Ruby `MESI_Two_Level` SLICC tables (D9). `L1cache.sm`'s enumerated transient states are the source for the `amber_pkg` state encoding. Every stable and transient transition is cross-checked against the executable protocol spec, not prose.

25.5 MonBus coverage

Every event class listed in Chapter 4.5 must be visible on MonBus in simulation. The `_monlite` wrapper is present-vs-absent in the regression so the gate-cost delta is part of the evidence.

26 Characterization

26.1 Board targets

amber is characterized on the same FPGA boards the repo uses for other components: Nexys A7 and Genesys 2. The default geometry (32 KiB / 64 B / 4-way) is expected to consume roughly 9 of the 135 36 Kb BRAMs on the Nexys A7, leaving headroom for the rest of the system.

26.2 Sweep dimensions

Table 6.0: Characterization sweep dimensions

Dimension	Values
Geometry	default (128 sets / 4 ways / 64 B), tiny (16 sets / 2 ways / 64 B), and boundary points from the 4–32 KiB / 2–8 way range
Replacement policy	LRU, tree-PLRU, FIFO, RANDOM
Rig	`amber` top, `amber_ace` top
Observation	`amber_monlite` present, `amber_monlite` absent

26.3 Reported metrics

For each point, the characterization records LUT, FF, RAMB36 (and RAMB18 where relevant), and fmax. The report follows the pumice pattern: a markdown table per board, a note on the critical path, and the observer-cost delta. A single characterization run is not the goal; the sweep is, because replacement policy and observation are the variables the research questions depend on.

26.4 Critical-path expectation

The expected critical path is through `amber_tag_array` port A: address decode → tag compare → hit/miss decision → data-array enable. The dual-port split (port B for snoops) removes snoop traffic from the CPU hit path, so the path should be comparable to a non-snoopy cache of the same geometry plus the CRRESP combinatorial decode in `amber_control`.

27 Future Hooks

27.1 Recorded future work, not blockers

The following items are explicitly out of scope for amber v1.0 but are recorded as hooks so the architecture does not accidentally close them off.

Table 6.1: Future hooks

Hook	Why it is future work	How amber leaves the door open
Lockup-free miss handling	The core research delta for `jet` (J1–J7)	`amber_core`'s module boundaries are clean; MSHRs replace the pending-fill bypass and the single-outstanding FSM without changing the array or snoop adapter interfaces.
MOESI O-state	Measurement for the pair rig, not a v1 requirement	The 3-bit state field in `amber_tag_array` already reserves the encoding headroom.
tree-PLRU cache_sim golden model	Timing-tight policy without a sim parity model today	`REPL_POLICY` is an elaboration parameter; adding a tree-PLRU model to cache_sim and the DV scoreboard is a model extension, not an RTL change.
STREAM attachment	First real consumer after TB masters (D10)	The CPU-side GAXI slave is the attachment point; no adapter needed.
Directory-lite snoop filter	Deferred from amber D4, reopened as onyx D3	The ACE-shaped snoop port is the boundary; a filter can be inserted between `amber_snoop_resp` and

Hook	Why it is future work	How amber leaves the door open
		the peer/onyx without touching `amber_core`.
ACE DVM and barriers	Out of scope per onyx D8	The snoop adapter and `amber_ace_issue` ignore these transaction types; adding them is an adapter/issuer change.
Exclusive loads/stores with internal monitor	Out of scope for v1.0; needed for atomic semaphore operations	Book §6.3 describes the internal monitor, §7.9–7.10 the competing-CU livelock scenarios. Reserved as a hook: adding exclusives requires the monitor plus the temporary snoop-stall safeguard recorded in the coherence FSM.
Clean-victim Evict messaging	Snoop-filter accuracy when a directory-lite filter is added (deferred from D4 to onyx D3)	Book §7.4 and §10.5: clean victims should send `Evict` so the filter can reset its occupancy bits. amber's eviction path currently issues `Evict` only on the ACE write master; this hook records that a separate clean-victim `Evict` message may be needed for filter accuracy.
Multi-copy-atomic store invariant	Documented assumption for v1.0; simplifies protocol reasoning	Book §14.33: making stores multi-copy atomic (all copies invalidated before the store is

Hook	Why it is future work	How amber leaves the door open
		visible) simplifies the design. amber assumes this invariant in its formal targets; if the assumption is ever relaxed, the coherence FSM and response ordering must be revisited.

27.2 Closing D3 and D11

D3 (memory-side interface) and D11 (array construction) remain open. Closing them will produce either a revised HAS or the MAS, depending on whether the decisions stay at architecture level or move into micro-architecture. Until then, the current directions — AXI4/ACE house wrappers and `sdpram_core`-based dual-port arrays — are the working defaults and the targets for the first RTL cut.

28 amber — Pre-Hardware Architecture Specification (Pre-HAS)

Version: 0.3 (draft, 2026-10-06) — **superseded by the full HAS (v1.0, same day)** once the PRD closed its rows (v0.5): the chapter book ratifies this sketch, and this file is kept as the decision-path record. **Status:** architecture sketch written while the PRD decisions are still OPEN. This document records **no decisions**. "PROPOSED" marks a working default put forward so the decision rows can be argued with concretely; a decision becomes DECIDED only when PRD.md records a name and a date. When the decisions close, each PROPOSED item here is either ratified into the full `amber_has` chapter book (the stream/andesite convention) or overridden and this file edited. v0.2 (2026-10-06): D2 landed in the PRD — CPU-side is a **GAXI slave**. F9 now answers probe-during-fill with a **pending-fill bypass** instead of an unbounded stall. Q1 answered — `amber_fabric` carries **both** rig attachments and the `RIG` elaboration parameter generate-selects. v0.3 (2026-10-06): Q1 re-answered by the owner — both rigs as **two tops**, `amber` (pair rig) and `amber_ace` (onyx rig), on a shared `amber_core`. No `RIG` parameter: the rig is whichever top you instantiate.

Governing documents: PRD.md (binding) · onyx PRD D7 (snoop-port contract) · jet PRD (what must stay constant) · GLOBAL_REQUIREMENTS.md · cache simulator (golden model)

28.1 1. Intent of this document

The PRD carries eleven decision rows and has closed three: **D2** (CPU-side is a GAXI slave), **D4** (snoop transport is ACE-shaped AC/CD/CR), and **D8** (observation through `*_monlite` wrappers only). The Pre-HAS makes the open rows concrete: for each, it sketches the feature the way a working default would build it, so the architecture can be reviewed before any RTL exists. It is deliberately cheap to change — one file, no chapter skeleton yet.

28.2 2. Feature sketches

28.2.1 2.1 Core datapath — the blocking bet

- **F1 — Blocking single-outstanding pipeline.** A miss freezes the front end until the fill completes and the original request replays. No MSHR, no hit-under-miss, no merge logic — that deletion is the entire point; it is jet's delta (jet J1–J5).
- **F2 — Geometry as elaboration parameters** (D1 OPEN). PROPOSED default profile 32 KiB / 64 B lines / 4 ways (128 sets) as the DV matrix centre of gravity; ranges 4–32 KiB, 32–64 B, 2–8 ways.

- **F3 — Pluggable replacement** (D7 OPEN). PROPOSED: true-LRU as the correctness reference (the cache simulator models it exactly), tree-PLRU as the FPGA-area variant; FIFO/random kept because the sim models them too. Policy is an elaboration parameter, never a code hack, so the sim↔RTL cross-check holds for every policy both sides implement.
- **F4 — Write policy** (D5 OPEN). PROPOSED: write-back + write-allocate — MESI's M state earns its keep, and dirty-victim handling is where blocking simplicity pays. Write-through/no-allocate retained as a bring-up mode because it deletes the victim path entirely.
- **F5 — Whole-line fills.** One burst per fill; sub-line fill granularity is a non-goal. Burst length derives from `LINE_BYTES / (BUS_WIDTH/8)`.

28.2.2 2.2 Coherence

- **F6 — MESI state per line** (D6 OPEN). PROPOSED plain MESI; MOESI's O-state recorded as a later elaboration upgrade, not v1 — ownership's win (dirty-transfer reduction between caches) is a measurement for the pair rig, not a v1 requirement. Insurance: allocate a 3-bit state field now (MESI needs 2; MOESI needs 3) so the upgrade is an encoding change, not an array rebuild.
- **F7 — ACE snoop responder** (D4 DECIDED — sketched, not re-decided). `amber_snoop_resp`: a thin adapter speaking raw ACE AC/CR/CD on the outside per the onyx D7 contract (IHI 0022 the definition) and driving a bus-agnostic probe interface inside. The cache core never contains ACE — only the adapter does; that is exactly why D4 was chosen (onyx, or a real ACE interconnect, can supersede the custom bus without touching cache RTL).
- **F8 — Snoop/local arbitration — the culsans problem.** Probes must not starve and CPU tag latency must stay bounded. PROPOSED: tag array on the dual-port `sdpram_core` — port A for CPU lookups, port B for snoop lookups; data array port A for CPU/fill, port B for snoop readout feeding
  CD. No arbitration stall in the common case; the *control* collision (miss FSM vs snoop FSM wanting the same array control port) resolved with snoop-priority round-robin inside `amber_control`.
- **F9 — Probe-during-fill, answered by a bypass** (direction set 2026-10-06: add the bypass). Blocking means a fill is usually in flight, and a probe against a pending line must not observe pre-fill state. The answer is not an unbounded stall: amber keeps one pending-fill register — single

outstanding by construction — recording {line address, state being installed, data availability}. A probe matching it is answered from the register: CRRESP reflects the post-fill state, and required CD beats are forwarded from the fill stream as its beats arrive. A probe that misses it takes the normal tag path. The snoop response is bounded either way. The correctness target sharpens rather than relaxes: *the bypass must be state-accurate* — a probe against a pending fill observes exactly the post-fill state, never pre-fill — alongside the response-bound liveness target. jet reopens the generalisation (jet J4/J6): multiple outstanding fills turn the register into a pending-fill table, which is jet's problem, not amber's.

- **F10** — **CRRESP response matrix** per MESI state, mirroring the RDS-DV ACE BFM default matrix as corrected in cocotb-framework 1.2.0 (MakeInvalid on M/O invalidates without data transfer — the bug the release fixed). The matrix ships as an explicit table in the full HAS.

28.2.3 2.3 Fabric side — where the two deployment rigs meet

- **F11** — **The two rigs.** *Pair rig* (amber D10): two ambers + shared memory, snoops peer-to-peer on the D4 port, misses are plain AXI4 to memory. *Onyx rig*: misses become coherent transactions toward onyx; onyx owns the memory side and drives the snoops. One cache core, two fabric attachments.
- **F12** — **Fabric side: two tops on one core** (Q1 answered 2026-10-06: do both, as two tops). The rig-agnostic cache core is shared; the fabric difference is confined to the top wrapper and its master stack:
  - `amber` — the pair-rig top. Misses are plain AXI4 to memory on the house `axi4_master_rd/wr` wrappers; snoop encodings never driven; the ACE snoop-responder port faces the peer cache.
  - `amber_ace` — the onyx-rig top. Fabric side is the existing `rtl/amba/ace/axi4ace_master_rd` / `axi4ace_master_wr` modules (ACE fields and RACK/WACK auto-pulse already in place), driven by a thin `amber_ace_issue` coherent-transaction issuer mapping cache events to the onyx D2 subset — read miss → ReadShared / ReadUnique (by desired exclusivity), write promotion → CleanUnique / MakeUnique, dirty eviction → WriteBack, clean eviction → Evict. Memory masters not exposed; the snoop-responder port faces onyx. No parameter selects between them: the rig is whichever top is instantiated. Both tops are compiled, and both are

exercised in the DV matrix — `amber` against a peer + memory environment, `amber_ace` against an onyx environment (§5).

- **F13 — Fill/drain engines** (D3 OPEN, direction: house wrappers). Fill = one AR burst, whole line. Drain = a small victim buffer (PROPOSED depth 1)
  - one AW burst. Single outstanding by construction — no reorder buffers.

28.2.4 2.4 Observation (D8 DECIDED — sketched, not re-decided)

- **F14 — `amber_monlite` wrapper.** MonBus event set: hit (read/write), miss with class split (cold/capacity/conflict, when the golden model supplies the class), snoop (type, hit/miss, response class), eviction (clean/dirty), MESI state transitions, fill/drain start+end. Tallied by `monbus_tally_axil` exactly like every other component; the wrapper never stalls the observed port — drop-and-count. The observer's own gate cost is measured present vs absent (F18), so "the observer does not perturb what it measures" is a quantified claim, not an assertion.

28.2.5 2.5 Verification & formal surface (D9 OPEN — direction proposed)

- **F15 — Pattern-B cocotb grids.** GATE/FUNC/FULL over the geometry × policy × rig matrix. Instruments already exist: the RDS-DV `AXI4ACESnoopMaster` BFM drives amber's snoop port and `AXI4ACEMasterRead/Write` drive the fabric side (cocotb-framework 1.2.0).
- **F16 — Golden-model parity.** Trace replay from `bin/apps/cache_sim/` at identical geometry + policy; hit/miss/miss-class counts must match exactly (PRD success criterion 1). The replacement policy parameter (F3) is what keeps this check meaningful across policies.
- **F17 — SymbiYosys targets** (PRD criterion 2): no stale data served after an external write (F9 ordering); no deadlock/livelock under fair arbitration. Proof surfaces named now so the RTL is built proof-shaped: `amber_control` FSM, `amber_snoop_resp`, tag-array port sharing, victim buffer handoff.

28.2.6 2.6 Cost & characterisation (PRD criterion 4)

- **F18 — Synthesis sweep.** Geometry params × replacement policy on Nexys A7 and/or Genesys 2; LUT/FF/RAM/fmax recorded in `docs/` the way pumice characterised. Includes the F14 observer-present vs absent delta.

28.3 3. Proposed block architecture

Proposed module set (`amber_*` prefix, per the cache-ip convention):

Module	Role	PRD anchor
`amber`	Pair-rig top: `amber_core` + plain AXI4 master stack + snoop responder toward the peer	D10
`amber_ace`	Onyx-rig top: `amber_core` + ACE master stack + coherent-transaction issuer + snoop responder toward onyx	D10, F12
`amber_core`	Shared rig-agnostic core, instantiated by both tops	—
`amber_cpu_frontend`	GAXI slave (D2 decided); request accept, response return, replay latch	D2
`amber_control`	Blocking pipeline FSM: lookup → hit / miss → evict → fill → replay; owns tag-array control arbitration (F8) and the pending-fill bypass register (F9)	D1, D5
`amber_tag_array`	`sdpram_core` tag + state store (3-bit state field, F6)	D11
`amber_data_array`	`sdpram_core` data store	D11
`amber_repl`	Pluggable replacement (policy elaboration parameter)	D7
`amber_victim`	Dirty victim buffer + drain trigger	D5
`amber_fill` / `amber_drain`	AXI4 read/write engines; plain `axi4_master_rd/wr`	D3

Module	Role	PRD anchor
	under `amber`, `axi4ace_master_rd/wr` under `amber_ace`	
`amber_snoop_resp`	ACE AC/CR/CD adapter; bus-agnostic inside — faces the peer in `amber`, onyx in `amber_ace`	D4
`amber_ace_issue`	Coherent-transaction issuer for the onyx D2 subset — `amber_ace` only	F12
`amber_monlite`	Observation wrapper (D8)	D8

Miss data flow: frontend accept → tag lookup (port A) → miss → `amber_repl` picks victim → dirty? `amber_victim` + drain engine WB → fill engine AR burst → data array write + state install → original request replays from the frontend latch → response.

Snoop data flow: AC arrives at `amber_snoop_resp` → tag lookup (port B) → line state determines CRRESP and whether CD beats follow → response driven. Conflict with an in-flight fill resolves per F9.

Observation taps sit on control-FSM decision points, not the datapath — the F8 port split means the datapath need not be touched to observe it.

28.4 4. Interface sketches (shapes, not pinouts)

Side	Sketch	Anchor
CPU-side	**DECIDED (PRD D2, 2026-10-06): GAXI slave** — house BFM/monitor coverage and skid/FIFO plumbing already exist, and STREAM can attach as first real consumer without an adapter. Must express single-beat reads, write-allocate fills, byte enables.	D2

Side	Sketch	Anchor
Fabric-side	Two tops, shared core (F12): `amber` = plain AXI4 masters to memory + AC/CR/CD snoop responder toward the peer; `amber_ace` = `axi4ace_master_rd/wr` + `amber_ace_issue` toward the CCU (onyx D2 subset: ReadShared / ReadUnique / CleanUnique / MakeUnique / WriteBack / Evict) + the same snoop responder toward onyx. The rig is the top you instantiate — no mode parameter.	D3, D4
Observation	MonBus through `amber_monlite`; no other debug surface.	D8
Register block	**None.** Parameter-only IP; all visibility through MonBus tallies, unlike every other component's APB CSR. Recorded explicitly so it is a decision and not an accident.	—

28.5 5. Parameter sketch (illustrative — feeds the DV matrix)

Parameter	PROPOSED default	Range / set
`SETS`	128	16–512
`WAYS`	4	2–8

Parameter	PROPOSED default	Range / set
LINE_BYTES	64	32–64
REPL_POLICY	lru	lru, tree_plru, fifo, random
WRITE_POLICY	wb_wa	wb_wa, wt_na
VICTIM_DEPTH	1	1–2
BUS_WIDTH	64	32–128

Rig is **not** a parameter: it is the top you instantiate — `amber` or `amber_ace` (F12). Derived: tag width = `ADDR_WIDTH - $clog2(SETS) - $clog2(LINE_BYTES) + 3` (state field, F6); fill beats = `LINE_BYTES / (BUS_WIDTH/8)`.

28.6 6. Open questions this sketch surfaces

#	Question	Where it resolves
Q1	~~One mode parameter `amber_fabric` vs separate adapters per rig~~ — **RESOLVED 2026-10-06:** two tops on a shared `amber_core` — `amber` (pair rig) and `amber_ace` (onyx rig); no rig parameter, both compiled, both in the DV matrix (F12)	Full HAS implements it
Q2	~~GAXI vs plain valid/ready CPU-side~~ — **RESOLVED by PRD D2 (2026-10-06, Sean): GAXI slave**	`amber_cpu_frontend` RTL
Q3	Geometry default vs Nexys A7 RAM budget (F2)	With the F18 cost sweep
Q4	3-bit state field headroom for MOESI (F6)	Cheap now, expensive later — recommend taking the insurance

#	Question	Where it resolves
Q5	~~Probe-during-fill stall-bound~~ — **RESOLVED 2026-10-06: add the pending-fill bypass** (F9); formal targets become bypass state-accuracy + response bound	Full HAS implements it; PRD criterion 2 depends on it
Q6	Replacement default: LRU reference vs tree-PLRU area (F3)	D7 closure, informed by F18

28.7 7. What the full HAS will add

The chapter book per the stream/andesite convention: front matter; purpose, conventions, definitions; use cases / key features / system context; graphviz block diagram; data flow; pin-level interfaces; performance targets; integration (clocking, reset, verification). The MAS follows when micro-architecture decisions land (array banking, pipeline staging). This Pre-HAS is the draft those books grow from — sections 2–6 above map onto chapters 2–5.

28.8 8. Cross-references

- amber PRD — binding requirements; decision record
- onyx PRD D7/D2 — snoop-port contract and coherent-transaction subset the ONYX rig speaks
- jet PRD — J-rows hold amber constant; nothing here may silently pre-decide jet (F9's stall policy is amber's; jet J4/J6 explicitly reopen it)
- References/AMBA_ACE_Interface_Definition.md and References/gem5-ruby-protocols/ — protocol definition and golden-protocol source
- RDS-DV cocotb-framework 1.2.0 ACE BFMs — the DV instruments F15 assumes

---

Last Updated: 2026-10-06 **Maintained By:** amber architecture (pre-decision working document)

29 jet — Product Requirements (DRAFT)

Version: 0.1 (draft, 2026-10-04) **Status:** scaffold only — inherits every DECIDED row of amber's PRD. Decided inherited rows so far: D8 observation (`*_monlite`, 2026-10-04) and D4 snoop transport (ACE-shaped AC/CD/CR per the onyx D7 contract, 2026-10-05). The rows below are the deltas jet must decide for itself; all are OPEN until they record a name and a date.

29.1 1. Purpose

The lockup-free (non-blocking) MESI snoopy L1 cache: amber with MSHRs. The research question is measurement — with coherence, arrays, and observation held constant, what do outstanding misses, hit-under-miss, and miss merging actually buy in latency and bandwidth, in simulation and on board?

29.2 2. Decisions jet adds to amber's

#	Decision	Candidates	What it drives
J1	MSHR organization	OPEN: full associativity keyed by address vs direct-mapped by line index vs CAM of pending lines; capacity 2–16 entries as an elaboration parameter	the miss queue, merge logic, formal surface
J2	Hit-under-miss scope	OPEN: hits to any resident line (default) vs hits only to lines not conflicting with a pending fill vs reads-only (writes still block)	hazard logic between fill port and hit port
J3	Miss merging policy	OPEN: merge only exact-line aliases (default) vs merge line + offset range	memory traffic, MSHR occupancy, fairness

#	Decision	Candidates	What it drives
		vs no merging (issue duplicate fetches, dedupe on return)	
J4	Ordering guarantees	OPEN: CPU-side responses in-issue-order vs out-of-order with ID tags; snoop-during-miss handling (a probe against a pending line must see the post-fill state or stall cleanly)	response path complexity, protocol proofs, consumer expectations
J5	Eviction under misses	OPEN: a dirty victim behind an outstanding fill must write back without deadlocking against the fill; victim buffer depth (0 = block like amber, 1–4 = decouple)	write-back path, the classic lockup-free deadlock corner
J6	Congestion/ fairness	OPEN: policy when MSHRs are full — stall new misses (strict) vs drop-and-retry vs throttle CPU-side accepts; plus snoop-vs-fill arbitration	starvation/ liveness proofs

#	Decision	Candidates	What it drives
		priority	
J7	The measured claim	OPEN: define the experiment now — same trace suite + same board as amber, report hit latency distribution, miss concurrency, bandwidth, and MSHR occupancy from MonBus capture	what “jet is faster by X” must mean, reproducibly

29.3 3. Success criteria (additions to amber’s)

1. **Correctness inheritance.** Every amber proof and golden-model parity result still passes with MSHRs present (proofs are re-run, not assumed).
2. **The delta, measured.** On the amber trace suite and at least one board: reported latency/bandwidth improvement with MonBus evidence, and the cache_sim cross-check extended so the model predicts jet’s concurrency numbers too (or the model gap is documented as a finding).
3. **The deadlock corner, proven.** J5/J6 formal targets: no deadlock when MSHRs are full and dirty victims wait behind fills.

29.4 4. Non-goals (for jet, beyond amber’s)

Speculative prefetching, adaptive replacement, quality-of-service policies, multi-banked parallelism beyond what MSHRs imply. Each is a future IP or a later decision.

30 onyx — Product Requirements (DRAFT)

Version: 0.4 (draft, 2026-10-11) **Status:** decisions pending — this page records the questions and the candidates, not answers. A decision becomes DECIDED when it records a name and a date, the way the reed-solomon PRD does it. Until then every row below is OPEN. **v0.4 adds only candidate analysis, recommendations, and PROPOSED invariants; nothing new is DECIDED.** v0.2: D7 decided 2026-10-05 (Sean) — onyx owns the ACE-shaped AC/CD/CR snoop port; amber/jet adapt to it behind a thin adapter. Decided together with amber D4, so the gate between the two is resolved. The sequencing constraint that remains: onyx still waits behind amber's pair rig (amber D10), its first consumer. v0.3: D10 direction tightened 2026-10-05 — the ACE BFM extension of the RDS-DV `axi4` family is now specified concretely (layering, four classes, build order); the row stays OPEN. RDS-DV implementation filed as RTLDesignSherpa-DV issue #86. v0.4: Chinnakonda book analysis folded in 2026-10-11 — Ch. 6/7/8/10/11/13/14 mined for candidate invariants, liveness scenarios, and scope-growth options. Book is cited in References; all new material is candidate/recommended/PROPOSED.

30.1 1. Purpose

A snoop-based cache coherency unit (CCU) in the shape of Arm ACE: the manager that sits between the repo's coherent caches and memory on the house AXI4 fabric. An attached cache (amber instance, jet instance) is an ACE *master*; it issues coherent transactions (ReadShared, ReadUnique, CleanUnique, MakeUnique, WriteBack, Evict) instead of managing peers directly. onyx receives those transactions, broadcasts a snoop (AC request) to the other caches, gathers their CR responses (DataTransfer / IsShared / PassDirty) and CD data, decides whether the transaction may be completed from cache-supplied data alone or memory must be read, and serialises same-address traffic. The research question is the manager side itself: what do fanout, gather, adjudication, and ordering cost — in gates, in latency, and in verification effort.

30.2 2. Background

ACE is AXI4 plus three snoop channels (AC, CR, CD) — and this repo already speaks AXI4 (skid buffers, GAXI FIFOs, `axi4_master_rd/wr` wrappers, Pattern-B BFMs). The natural-protocol argument: every piece of onyx's data path reuses existing, proven plumbing; the new logic is the coherence decision layer and the serializer. The definition is Arm IHI 0022 (the ACE chapters); the worked example is `pulp-platform/culsans` (ETH/PlanV), whose two halves are exactly the reading list:

- **culsans' CVA6 coherent WB cache** — how a cache becomes a snoop responder: the AC/CD/CR channels, how snoop tag lookups arbitrate with the core's own tag port, and the MOESI-ish line states.
- **`ace_ccu_top`** — the manager: broadcast snoops on a coherent read/write, collect responses, decide whether memory is read, and serialise same-address transactions.

Treat culsans as a worked example, not a dependency (last push 2024, lightly maintained). Note also what onyx is *not*: it is not a directory, not an L2, and not a general NoC — amber's deferred "snoop filter / directory-lite" question (amber D4) shows up here only as one candidate row (D3).

Repo anchors the design must respect: resets on the `ALWAYS_FF_RST` macros, no bespoke SRAM (shared `sdpram_core` / house FIFO primitives), `[DEPTH]` array syntax, valid/ready streaming contracts, MonBus observation through `*_monlite` wrappers only (amber D8's decision — the observer must never stall the measured paths), filelists registry, and GLOBAL_REQUIREMENTS.

30.3 3. Decisions that pick the design

#	Decision	Candidates	What it drives
D1	Functional scope	OPEN: CCU only — N ACE slave-side ports toward caches, one AXI4 master toward memory, one ACE-Lite IO port (the culsans shape; recommended starting scope) vs CCU + address striping / multiple memory ports (interconnect territory; non-goal today)**Book input:** §10.1 (banks ≥ master count), §10.2–10.3 (centralized vs. distributed manager; BIU as mapper), §10.7 (coherency logic can live in L2 or BIU). Confirms onyx as the	block boundary, what "onyx done" means

#	Decision	Candidates	What it drives
		BIU/manager block rather than an L2; keeps CCU-only the recommended v1 scope.	
D2	ACE transaction subset	OPEN: research subset — ReadShared / ReadUnique / CleanUnique / MakeUnique / WriteBack / Evict (recommended) vs subset + barrier support vs full ACE incl. DVM**Book input:** §7.2–7.4 give the full ARSNOOP/AWSNOOP taxonomy; WLU/WU are entire/partial-line writes used for non-temporal/all-bytes-updated stores and coherent mem-copy (§11.1); WC/WE are clean-victim/LLC-victim writes. The recommended research subset stays	front-end decoder, BFM scope, conformance matrix

#	Decision	Candidates	What it drives
		RS/RU/CU/MU/WB/Ev; WLU/WU/WC/WE are recorded as scope-growth option N5.	
D3	Snoop fanout	OPEN: broadcast to all attached caches (culsans shape, simplest, recommended for v1) vs directory-lite snoop filter as an elaboration parameter (amber D4's deferred question, reopened here) vs snoop filter mandatory**Book input:** §10.3–10.5 describe inclusive-directory options: copy-of-L1-tags, L1ID sharer vector in tag, or separate snoop filter sized ≈1.5× combined L1. §10.6.2 notes the false-positive-snoop corner (multiple shared responders → pick one; many	fanout logic, gate count, the coherence-traffic measurements

#	Decision	Candidates	What it drives
		systems read DRAM for shared copies). Broadcast remains recommended v1; directory-lite is a future elaboration parameter.	
D4	Response gather	OPEN: fixed response window (all caches respond within N beats, presumed-miss after timeout — simple, needs a proven bound) vs tagged outstanding snoop IDs with per-transaction completion (exact, more bookkeeping)**Book input:** §7.1 and §7.7 state snoops have no IDs and must be responded to in issue order; §14.18 confirms the three snoop channels are independent except for the same-address WB/Ev/WC/WE	gather buffer, formal surface, ACE conformance

#	Decision	Candidates	What it drives
		dependency. §7.7 gives the response bound: a snoop may be delayed only until the last data beat of a matching fill has arrived, otherwise it must respond not-present; holding it beyond that deadlocks.	
D5	Same-address serialization	OPEN: per-address scoreboard allowing interleaving of different addresses (recommended; the interesting manager) vs full in-order single outstanding (simplest, strangles concurrency; fine as a bring-up mode)**Book input:** §7.5, §10.7, §10.8 describe same-AxID ordering and the banked PoS with per-address linked lists (older	the serializer, the deadlock/liveness proof, jet J4 interactions

#	Decision	Candidates	What it drives
		completes first, different addresses any order); §10.9.1 notes device accesses routed to a dedicated bank for program order; §10.8 gives outstanding-access sizing math (≈40 read + 40 write transactions for 200-cycle DRAM latency on a 1 GHz 16-byte bus). Informs the scoreboard/serializer design but does not decide it.	
D6	Memory-read elision	OPEN: full adjudication — read memory only when no cache supplied data and no PassDirty (correct ACE semantics; recommended) vs always-read simplification for bring-up**Book input:** §10.5–10.6.2 refine adjudication: when multiple	memory traffic, the measurable win onyx exists to quantify

#	Decision	Candidates	What it drives
		DataTransfer responders exist, the manager must pick one; many real systems read DRAM for Shared copies and accept data only from an O-state holder; MOESIF Forward state is one way to designate a single forwarder. Therefore “any DataTransfer elides memory” is NOT the safe rule — elision requires a dirty (PassDirty or O-state) responder, or a single designated forwarder.	
D7	Cache-side interface ownership	**DECIDED 2026-10-05 (Sean): onyx defines and owns an AC/CD/CR-shaped snoop port; amber/jet stay bus-agnostic internally and add a thin adapter to it.** Chosen over caches natively	the amber D4 ↔️ onyx contract, who changes when ACE supersedes the custom bus

#	Decision	Candidates	What it drives
		growing ACE ports — the cache cores stay unpinned from ACE except at the adapter, so onyx (or a real ACE interconnect, e.g. culsans' `ace_ccu_top`) can supersede the custom bus without touching cache RTL.	
D8	Domain / barrier rules	OPEN: single shareability domain, barriers ignored (recommended non-goal, recorded here so it is a decision and not an accident) vs minimal barrier ordering vs full DVM**Book input:** §6.4–6.5 and §14.33 recommend making stores always multi-copy atomic even though Arm does not require it; this is the book's justification for	scope honesty, later upgrade path

#	Decision	Candidates	What it drives
		terminating barriers in the processor and ignoring them at the manager. Single shareability domain / barriers ignored remains recommended.	
D9	Observation	OPEN (direction: amber D8 pattern): `*_monlite` taps for coherent-txn start, snoop fanout per transaction, response gather completion, memory-read elision hit/miss, serialization stalls, and WB/Evict drains; tallied by `monbus_tally_axil`**Book input:** §8.2/§8.3/§10.6 treat independent snoop paths and fairness as first-class observables; §10.8 gives outstanding-access sizing math	perf counters, board capture, DV scoreboard

#	Decision	Candidates	What it drives
		that shapes the counters listed above.	
D10	Verification strategy	OPEN (direction: Pattern-B cocotb gate/func/full. **ACE BFM — an extension of the `axi4` BFM family in RDS-DV, layered the way that family is built** (analysis from inspecting `CocoTBFramework/components/axi4/`): (1) field configs — an `AXI4ACEFieldConfigHelper` adds `ARSNOOP[3:0]` / `AWSNOOP[2:0]` to the AR/AW field configs; (2) signal-map patterns — new `axi4ace_*` protocol patterns (decide there: framework-shipped vs. user-loadable, the signal_mapping_helper question); (3) transaction/packet layer — the six	DV structure, formal areas, CI time

#	Decision	Candidates	What it drives
		D2 transaction types as an enum, CRRESP as a bitfield (DataTransfer/Error/PassDirty/IsShared/WasUnique); (4) four BFM classes — `AXI4ACEMasterRead` / `AXI4ACEMasterWrite` as subclasses (snoop type on commands; the BFM **drives RACK/WACK** when playing master against an onyx-port DUT, since the RTL auto-pulse in `axi4ace_master_rd/wr` only covers DUT-as-master), `AXI4ACESnoopSlave` (new, no base: receives AC, drives CR/CD in **issue order** — snoops carry no ID — with programmable cache-state behavior producing the correct CRRESP;	

#	Decision	Candidates	What it drives
		this is the golden-model hook that replays gem5 traces through amber), `AXI4ACESnoopMaster` (new: drives AC, gathers CR/CD in order — drives onyx's fanout in DV); (5) compliance checker — in-order CR/CD, no snoop to the requesting master, CD-before-CDLAST, CRRESP validity per snoop type; (6) main-repo TB classes under `bin/TBClasses/axi4ace/` per consumer. **Build order:** snoop-slave BFM first (amber's pair rig needs it), snoop-master BFM lands with onyx; ships as an RDS-DV framework release the way the irq BFM did. Python golden	

#	Decision	Candidates	What it drives
		model derived from the gem5 Ruby `MESI_Two_Level` spec in `../References/gem5-ruby-protocols/`; the ACE subset contract is the in-repo definition doc; SymbiYosys proofs of same-address serialisation and no-deadlock under fair arbitration; culsans behaviours as informal cross-checks, not oracles)**Book input:** §7.6–7.10, §8.1–8.3, §10.9 give concrete liveness scenarios: WLU-vs-WB valid/ready deadlock when SI buffers are full (§7.6.1/§7.7.1); CU livelock on RU-then-downgrade (§7.8, §10.9); store-exclusive	

#	Decision	Candidates	What it drives
		semaphore livelock and POS-monitor recipe (§7.9–7.10); starvation from fixed priority → timeout fairness (§8.3). These become the named proof obligations under Success criterion 3.	
D11	Master count & profile	OPEN: N coherent cache masters 2–4 as an elaboration parameter (culsans uses 2–4 CVA6) vs fixed 2 for the research rig; line size / profile locked to amber's geometry vs independently parameterised**Book input:** §10.1 states banks should be ≥ master count (4 CPUs → at least 4 banks). Supports keeping N=2–4 as an elaboration parameter and sizing internal resources	array dimensions, DV matrix, the measured system's realism

#	Decision	Candidates	What it drives
		accordingly.	
N1	RACK/WACK omission-by-contract	OPEN: onyx owns the snoop-port shape (D7), so it can define data/snoop-path ordering inside its domain and omit RACK/WACK from the onyx↔️cache contract. Book §7.20 and §14.39 say RACK/WACK are not required if data and snoop paths are ordered and not buffered past the coherency point. This conflicts with the current family convention in the ACE definition doc, which auto-pulses RACK/WACK at the adapter; the definition doc wins as the current contract, and the book's claim is recorded here as the candidate rationale for omission.	adapter logic, snoop/data ordering assumptions

#	Decision	Candidates	What it drives
N2	AxLock / PoS monitor for cross-master exclusives	OPEN: arm a POS monitor on AxLock for exclusive transactions: one master receives EXOKAY, competing CUs receive OKAY (fail), and invalidation snoops are withheld from the winning master until its store completes. Book §7.9–7.10 and §14.36 describe this as the livelock-reduction recipe for store exclusives. Out of the initial D2 subset; recorded as a candidate upgrade.	exclusive-transaction handling, POS monitor logic
N3	Snoop-response sourcing from in-flight fills	OPEN: a snoop matching an in-flight fill must respond not-present if no data beat has arrived, and may forward from the fill buffer only after the last data beat	snoop responder, fill-buffer comparators, formal progress proof

#	Decision	Candidates	What it drives
		has arrived; the response must never depend on an unrelated event. Book §7.7, §8.2, §10.6. Elevates the D4 book input to a proposed invariant.	
N4	CD-path independence	OPEN: never share the snoop-data return path (CD) with WDATA; reserve credit/capacity for the WB/Ev/WC/WE family so snoop responses cannot be blocked by ordinary writes. Book §10.6 and §14.11–14.13 emphasize an independent snoop path as the deadlock-avoidance bedrock. (ACE permits CDDATA to be combined with WDATA; the book argues against it.)	snoop data channel, write-buffer allocation, credit logic
N5	WLU/WU +	OPEN: add	front-end

#	Decision	Candidates	What it drives
	WC/WE scope-growth option	WriteLineUnique / WriteUnique for coherent DMA/mem-copy, plus WriteClean / WriteEvict for clean-victim handling. Book §7.3–7.4 and §11.1–11.3 tie WLU/WU to mem-copy traffic (read stream → RS+Evict, write stream → WLU/WU); WC/WE handle clean victims and LLC victim caching. Not in the recommended D2 subset; recorded as a scope-growth candidate tied to a future coherent DMA/mem-copy use-case.	decoder, fanout/adjudication, conformance matrix
N6	DVM upgrade path	OPEN: if DVM is ever promoted, follow the book's recipe: DVM Messages are two-beat read transactions sent back-to-back with	DVM message handling, serialization, drain logic

#	Decision	Candidates	What it drives
		no intervening transaction; one DVM Sync per cluster is serialized at the SI; DVM Complete uses a unique ARID and is sent only after all translated-address buffers have drained. Book §7.16 (TLB maintenance / DVM). Revisits the D8/DVM non-goal but does not promote it.	
N7	Prefetch-traffic handling constraint	OPEN: the CCU must never see speculative store-ownership requests; prefetch traffic, if it reaches onyx at all, is speculative read-shared/read-clean only. Book §13.3: processors do not prefetch for stores because prefetches are speculative and must not make speculative state changes or set	prefetch filtering, BFM scenario matrix, coherency checker

#	Decision	Candidates	What it drives
		Dirty bits.	

30.4 4. Design invariants (proposed)

The following bedrock liveness rules are mined from Chinnakonda Ch. 7/8/10/14. Each is **PROPOSED** pending an owner ruling; none is DECIDED in v0.4.

1. **Snoop forward progress (PROPOSED).** A snoop may wait only on (a) a same-address outstanding WB/Ev/WC/WE, or (b) a guaranteed-to-complete temporary block (e.g. a retired store completing after it has received ownership). It must never wait on an unrelated transaction or an unpredictable buffer-drain event.
2. **Snoops ignore WLU/WU entirely (PROPOSED).** §7.13: a snoop to the same address as a WLU/WU is always allowed to make progress; the WLU/WU does not block it.
3. **Snoop responses never depend on WU/WLU completion (PROPOSED).** Otherwise SI buffers full of interdependent WU/WLU traffic can deadlock.
4. **CD never blocks on WDATA (PROPOSED).** The snoop-data return path must have independent forward progress; if it shares resources with ordinary writes, those resources must be credit-reserved for the WB/Ev/WC/WE family.
5. **Multi-copy-atomic stores (PROPOSED).** A store is considered complete only after all ownership invalidations have been performed everywhere; this is the book's recommended simplification that makes barriers-ignored at the manager defensible.
6. **CR before CD, CD-before-CDLAST, in-order CR/CD (PROPOSED).** A snoop response indicates whether data will follow; data beats are in order and CDLAST marks the final beat; CR and CD keep the issue order of the ACs.
7. **No snoop to the requesting master (PROPOSED).** Already a D10 compliance checker rule; restated here as a design invariant.

30.5 5. Success criteria

1. **Three-block coherence.** Two amber instances (or amber + jet) plus onyx form a coherent system that passes the golden-model parity suite amber D9 defines — the pair rig grown by one block, same traces, same policy space.

2. **IHI 0022 conformance on the D2 subset.** A scoreboard checks the AC/CR/CD behaviour per transaction type: the CR bits (DataTransfer, IsShared, PassDirty) of every gather are correct for the transaction and the caches' states.
3. **The serializer, proven.** Formal: no two same-address coherent transactions proceed incompatibly; no deadlock under fair arbitration at N ∈ {2, 3, 4} masters; the D4 response-gather mechanism cannot livelock (a presumed-miss bound exists and is proven). The Chinnakonda book supplies the named proof obligations: absence of CU livelock at the PoS, no WLU/WB-style deadlock in the fanout/gather path, and fairness (no fixed-priority starvation) under N ∈ {2, 3, 4}.
4. **Observation.** Every D9 event class visible on MonBus in simulation and captured on at least one FPGA board.
5. **Cost.** Synthesis/implementation numbers (LUT/FF/RAM, fmax) on Nexys A7 and/or Genesys 2, recorded in `docs/` the way pumice characterised.

30.6 6. Non-goals

Cache storage in onyx (it is a manager, not an L2 — tempting because a CCU's gather point is the natural place to cache; explicitly out), directory-based coherence (the D3 snoop-filter candidate is the closest this IP comes), ACE DVM and full barrier semantics (D8; revisited by candidate N6 as a future upgrade path, not promoted out of non-goals), cache stash/dealloc features, multi-CCU / multi-domain systems, and any CPU-side or upstream-master port beyond ACE-Lite IO. Each is a future IP or a future decision, not an onyx requirement.

30.7 7. Where the block sits

The fabric's coherence point, on three kinds of port: **cache-side**, N ACE-facing ports accepting coherent transactions from amber/jet instances acting as ACE masters (D7, decided 2026-10-05: onyx owns this port's ACE shape; caches adapt); **memory-side**, one AXI4 master toward the house memory path (D6 fixes when it is read); **inter-cache**, the snoop fanout — AC requests driven into each peer cache, CD snoop data and CR snoop responses returned. No CPU port, no register block beyond what observation needs (D9 tallies live in `monbus_tally_axil` like every other component). Standalone-testable with BFM caches long before amber exists: culsans' `ace_ccu_top` demonstrates exactly that decoupling.

30.8 8. References

1. Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022 (ACE chapters) — the normative definition. The in-repo working subset is recorded in `../References/AMBA_ACE_Interface_Definition.md`.
2. Murali Chinnakonda, *Designing a Multiprocessor Cache Subsystem* — Arm-ACE-centric multiprocessor cache subsystem design; ch. 7/8/10/14 mined 2026-10-11. (Source markdown in the owner's calibre library; not mirrored in-tree.)
3. `pulp-platform/culsans` — `rtl/src/ace_ccu_top.sv` and the CVA6 coherent cache: a working ACE CCU and a working snoop-responder cache. Worked example, not a dependency. https://github.com/pulp-platform/culsans
4. *A Primer on Memory Consistency and Cache Coherence* (Sorin, Hill, Wood, 2nd ed., stored alongside the ACE definition doc) — coherence-theory background the transaction types instantiate.
5. gem5 Ruby SLICC specs (`gem5-ruby-protocols/`, stored alongside the ACE definition doc) — executable transient-state tables for the same protocol family; the golden-model source material, phrased in messages rather than ACE signals.
6. amber/jet improvement plan memo — future family-level planning document at `../docs/amber-jet-improvement-plan.md`; linked for cross-reference even though it does not yet exist.

31 AMBA ACE — Working Interface Definition for cache-ip

Distilled 2026-10-05 for the `cache-ip` family, then **verified clause-by-clause against IHI 0022H on 2026-10-05** (Arm's direct PDF, linked in §7; fetched to `/tmp` for comparison, not stored in-tree). Verification confirmed every transaction type, channel payload, CRRESP bit, and the ACE-Lite channel exclusions; it caught and fixed two errors in the ordering rules (the original draft claimed ACE serializes snoops one-at-a-time and mandates CD-before-CR — ACE requires only per-channel ordering and permits multiple outstanding snoops; the stricter behavior is recorded above as our own adapter convention). This document is the in-repo definition of the **ACE-shaped port contract** referenced by amber D4 and onyx D7 (family decision 2026-10-05: cache-ip is ACE-shaped).

Normative reference: Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022 (ACE chapters). Available from Arm without charge (direct PDF in §7); Arm copyright, so deliberately not mirrored here. Where this document and IHI 0022 disagree, IHI 0022 wins — file a docs bug.
Worked example: `pulp-platform/culsans` (`ace_ccu_top`, Solderpad license) — read its RTL after this page, not before.

31.1 1. What ACE is

ACE (AXI Coherency Extensions) is AXI4 extended so that multiple caching masters can share memory coherently. An ACE master is typically a CPU's cache; an ACE-Lite master is a non-caching agent (DMA, IO) that wants coherent *access* without being snooped. Between the masters and memory sits a coherency manager — in this family, **onyx** — which receives coherent transactions, broadcasts snoops to the peer caches, gathers their responses, decides whether DRAM must be read, and serializes same-address traffic.

ACE adds three channels to AXI4. Everything else — AW/W/B/AR/R semantics, burst rules, handshake rules — is unchanged AXI4 and is not restated here.

31.2 2. Channel inventory

31.2.1 2.1 Coherent master → manager (front side, per master)

The five AXI4 channels, with a transaction-type field added to each address channel:

Channel	New/changed payload	Notes
AR + R	`ARSNOOP[3:0]`	read-side coherent transaction type
AW + W + B	`AWSNOOP[2:0]`	write-side coherent

Channel	New/changed payload	Notes
		transaction type

Data and response channels are plain AXI4. A coherent read (e.g. ReadShared) delivers its data on the ordinary R channel, sourced by the manager from a peer cache or from memory as the adjudication decides.

Full ACE master interfaces also carry two acknowledge handshakes outside the five channels: **RACK** (read acknowledge — master pulses after the last read data beat of each transaction) and **WACK** (write acknowledge — master pulses after the write response). ACE-Lite excludes both (IHI 0022H §D11.1). Our D2 subset needs no ordering semantics from them; the family convention is to auto-pulse them promptly at the adapter and record that choice in the module docs.

31.2.2 2.2 Manager → cache, cache → manager (snoop side, per cache)

Three new channels. Direction below is from the point of view of the cache (amber/jet); the manager (onyx) sees the mirror image.

Channel	Dir	Payload	Meaning
AC — snoop address	in	`ACADDR`, `ACSNOOP[3:0]`, `ACPROT[2:0]`	"What state does this line have in you, and what should you do about it?"
CR — snoop response	out	`CRRESP[4:0]`	the answer: will you transfer data, is the line shared, is dirty responsibility passing
CD — snoop data	out	`CDDATA`, `CDLAST`	the line's data, when CRRESP said it would come

Handshake rules mirror AXI4 (VALID/READY, no combinatorial dependency of READY on VALID). Ordering rules that matter (IHI 0022H §D3.8–§D3.9, §D5.2, §D6.2):

- **CR and CD only after AC.** The snoop response and any snoop data must only be asserted after the ACVALID/ACREADY handshake.
- **In-order responses, in-order data.** CR responses must be returned in the same order as the AC addresses; CD data likewise. The spec permits a cache

to have **multiple outstanding snoop transactions** — serialization is not required by ACE, only per-channel ordering is.

- **Family convention (decided with onyx D4):** the amber/jet adapter completes a transaction's CD beats before asserting its CR, so onyx's gather sees one fully-closed response at a time. This is stricter than ACE requires and simpler to verify; it is our contract, not the spec's.
- **Snoop vs. local access arbitration.** A snoop reads the tag array and may evict/change a line's state, so it must arbitrate with the cache's own CPU-side tag port. This arbitration point is the classic cache-side cost of becoming a snoop responder (culsans' CVA6 cache shows one arrangement).

31.3 3. Transaction types (the onyx D2 research subset in bold)

Read side (`ARSNOOP`): ReadOnce, **ReadShared**, **ReadUnique**, ReadClean, ReadNotSharedDirty, **CleanUnique**, **MakeUnique**, plus barrier/DVM forms (out of scope, onyx D8).

Write side (`AWSNOOP`): **WriteUnique**, WriteLineUnique, **WriteBack**, WriteClean, **Evict**, plus barrier/DVM forms (out of scope, onyx D8).

The six bolded transactions are the subset onyx D2 recommends; the definition of the subset is: everything amber or jet needs to run MESI write-back coherence against a manager, and nothing that requires DVM or barrier semantics.

Intent of each subset member, one line each:

- **ReadShared** — fetch a line to read; other caches may keep Shared copies; the line arrives in S (or E if nobody shared it).
- **ReadUnique** — fetch a line to write; all other copies must be invalidated; if a peer held it dirty, its data (or DRAM's) is supplied.
- **CleanUnique** — "I already hold the line; make my copy Unique." No data transfer; peers with Shared copies drop them.
- **MakeUnique** — like CleanUnique but the requester does not need the data (it will overwrite the whole line); peers invalidate.
- **WriteBack** — retire a dirty line to memory; the manager must merge the data into DRAM (or hand it to a subsequent reader).
- **Evict** — retire a clean line; a hint: no data, no DRAM write needed.

31.4 4. The CRRESP decision bits

CRRESP is where a snooped cache answers. The four bits the manager's adjudication logic consumes (bit 4, `WasUnique`, also exists per IHI 0022):

Bit	Name	Meaning when set
[0]	DataTransfer	"I will drive the line's data on CD." Required of a Modified responder (or any responder told to transfer).
[1]	Error	"My lookup had an error." Propagated, not interpreted.
[2]	PassDirty	"The line is/was dirty and dirty responsibility moves to whoever receives the data" — set with DataTransfer when the responder passes dirty data, and on its own when the data is already moving elsewhere. The manager uses it to decide whether DRAM must be (re)written.
[3]	IsShared	"Another master (or I) may hold a copy." Tells the manager the requester must not be given Exclusive/Unique state, and that DRAM-read elision is safe when data was supplied.

The adjudication onyx D6 specifies falls straight out of these bits: a coherent read completes from cache-supplied data alone iff some CR carried `DataTransfer`; DRAM must be read iff none did

and no `PassDirty` moved dirty responsibility; DRAM must be written iff `PassDirty` passed dirty data that the requester did not absorb.

31.5 5. ACE-Lite

ACE-Lite is the non-snooped half: a master may issue coherent transaction types but never receives AC, so it never drives CR/CD. onyx's IO port (onyx D1) is ACE-Lite — this is why the earlier "ACE-lite-shaped snoop channel" candidate in amber D4 was set aside: a snoop *responder* must drive CR/CD, and ACE-Lite has no snoop channels at all.

31.6 6. What this family implements, and what it deliberately does not

In scope (amber D4 / onyx D2 subset, single shareability domain, onyx D8): the six transaction types of §3, the AC/CR/CD contract of §2, the CRRESP semantics of §4, broadcast snoop fanout (onyx D3 v1), same-address serialization (onyx D5).

Transport for this contract exists in `rtl/amba/ace/` (added 2026-10-05): `axi4ace_master_rd/wr` and `axi4ace_slave_rd/wr` (front-side movers with `ARSNOOP`/`AWSNOOP`, auto-pulsed `RACK`/`WACK` on the masters), the `axi4ace_snoop_slave` / `axi4ace_snoop_master` AC/CR/CD movers, and `rtl/amba/monitor/axi4ace_snoop_monitor_lite` for snoop-side observation.

Out of scope, recorded as decisions and not accidents (onyx D8): DVM (distributed virtual memory) messages, barrier ordering, multiple shareability domains, stash/dealloc hints. The repo's port contract is **ACE-shaped** — same channels, signals, payload fields, and ordering rules on the pins — without claiming whole-spec conformance; see onyx D7 for why "shaped" rather than "compliant."

31.7 7. Sources

1. Arm, *AMBA AXI and ACE Protocol Specification*, IHI 0022, ACE chapters — the definition. Current issue H (2020), direct PDF: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/IHI0022H_amba_axi_protocol_spec.pdf (landing page: https://developer.arm.com/documentation/ihi0022/). Available from Arm without charge; Arm copyright — cited, not mirrored. Sections this document was verified against: D2.2 (channels), D3.8–D3.9 (signaling, dependencies), D3.20–D3.22 (CRRESP), D5 (snoop transactions), D6.2 (sequencing), D11.1 (ACE-Lite).
2. `pulp-platform/culsans` — `rtl/src/ace_ccu_top.sv` and the CVA6 coherent cache: a working ACE CCU and a working snoop-responder cache.

Worked example, not a dependency.
https://github.com/pulp-platform/culsans

3. *A Primer on Memory Consistency and Cache Coherence* (Sorin, Hill, Wood, 2nd ed., stored alongside this file) — the coherence-theory background the transaction types above instantiate.
4. gem5 Ruby SLICC specs (`gem5-ruby-protocols/`, stored alongside) — executable transient-state tables for the same protocol family; the golden-model source material, phrased in messages rather than ACE signals.

32 gem5 Ruby protocol specs (reference extract)

Executable, state-table-style cache-coherence protocol specifications in gem5's SLICC language, extracted verbatim from gem5 `master` at commit `f5c5a6e390f55dd5984977815bf9d0bd05da6945` (2026-09-07) via sparse clone.

Two protocols, as requested for the cache-ip research library:

- **MESI_Two_Level** — classic snoopy MESI with a shared L2. The L1 cache controller (`MESI_Two_Level-L1cache.sm`) is the closest executable model to amber: every transient state (IS, IM, SM, S^I, M^I, …) between a stable MESI state and a pending response is enumerated as an explicit state with its own transition table. `MESI_Two_Level-msg.sm` is the message-type contract (the shape a D4 snoop transport must express); `MESI_Two_Level-dir.sm` + `-L2cache.sm` show the point where a passive memory-side agent has to make coherence-visible decisions.
- **MOESI_CMP_directory** — the directory-based contrast set: what changes when a home/directory agent (not a broadcast medium) owns coherence serialization. Useful for ruling D4 options in or out with a working executable spec instead of prose.

Why these exist here: the Primer's MESI diagrams show only stable states; most coherence bugs live in the transient states. These files are the enumerated transient-state tables — the raw material for deriving amber's `amber_pkg` state encoding and for building the Python reference model the DV scoreboard replays against (amber D9, jet J7 cross-checks).

License: each `.sm`/`.slicc` file carries its complete copyright and BSD-3-clause license header inline (ARM Limited; Mark D. Hill and David A. Wood) — redistribution terms are inside the files themselves. `LICENSE` is gem5's root file, an unrendered BSD-3 template, included as-shipped.

Upstream: https://github.com/gem5/gem5 (`src/mem/ruby/protocol/`). This directory is a read-only extract; track upstream for changes.

33 Cache Associativity Simulator

A standalone, trace-driven cache simulator for exploring set count, way count, block size, and replacement policy. Written in vanilla HTML5/CSS/ES5-ish JavaScript with no build step; open `index.html` directly as `file://`.

33.1 Files

```
bin/apps/cache_sim/
  index.html          -- UI
  style.css           -- mobile-first styles
  js/model.js         -- pure simulation model (no DOM)
  js/app.js           -- DOM glue, generators, and guided example
  test/run_tests.js   -- zero-dependency TAP-ish node test runner
  README.md           -- this file
```

33.2 Model API

```
var CACHESIM = require('./js/model.js');

var result = CACHESIM.simulate(
  { sets: 4, ways: 1, blockSize: 1, policy: 'LRU' },
  [0x00, 0x10, 0x20, 0x00],
  1   // seed for RANDOM policy / reproducibility
);
```

`simulate(config, addresses, seed)` returns:

```
{
  config: { sets, ways, blockSize, policy },
  perAccess: [
    { addr, hit, setIndex, way, missClass }, ...
  ],
  totals: {
    hits, misses, hitRate,
    compulsory, capacity, conflict
  },
  occupancy: [ [tagOrNull, ...], ... ]   // sets x ways
}
```

`missClass` is one of `'hit'`, `'compulsory'`, `'capacity'`, or `'conflict'`. Capacity classification is computed by running a shadow fully-associative cache with the same total size and policy.

Other model helpers:

- `CACHESIM.validateConfig(config)` -> `{ok, errors}`
- `CACHESIM.parseAddressText(text)` -> address array (NaN for bad lines)

- `CACHESIM.generateSequential(length, start)`
- `CACHESIM.generateStrided(length, start, stride)`
- `CACHESIM.generateRandom(length, seed, maxAddr)` (default maxAddr = 256)

33.3 Running tests

```
node bin/apps/cache_sim/test/run_tests.js
```

Exit code is 0 when all tests pass.

33.4 UI features

- Sliders for sets, ways, and block size; dropdown for policy.
- Address trace textarea plus one-click sequential, strided, and random generators.
- Big-number hit rate, totals table, per-access log, and per-set occupancy.
- Guided example button that loads a cyclic trace and compares direct-mapped vs 2-way set-associative behavior.